

STUDY GUIDE · FROM DOCKERFILE TO DOCKER COMPOSE TO KUBERNETES

Multi-Stack Applications on Kubernetes

From Docker Compose to Kubernetes

Seven applications in seven stacks (React, Node.js, Python, Go, Java with Spring Boot, PHP with Laravel and plain JavaScript), packaged as Docker images, run together with Docker Compose and deployed to Kubernetes. 15 lessons: the architecture, Dockerfiles across stacks, images and registries, Compose, the mapping from Compose to Kubernetes, ConfigMaps and Secrets, health probes, resources, storage, networking and Ingress, scaling and rollbacks, debugging, Jobs and CronJobs, multi-container Pods; plus the service contract, the troubleshooting method, the capstone, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-multi-stack-kubernetes>

Contents

Study material

- 01 · The Bookshop architecture
- 02 · Why multiple technology stacks?
- 03 · Dockerfiles across stacks
- 04 · Images, tags and registries
- 05 · Docker Compose
- 06 · From Docker Compose to Kubernetes
- 07 · ConfigMaps and Secrets
- 08 · Health probes
- 09 · Resources: requests and limits
- 10 · Storage and databases
- 11 · Services, networking and Ingress
- 12 · Scaling, rolling updates and rollbacks
- 13 · Logging and debugging
- 14 · Jobs and CronJobs
- 15 · Multi-container Pods

Service contract

Troubleshooting labs

Capstone · the complete multi-stack platform

Glossary

Interview questions

Each lesson explains one part of taking an application from source code to Kubernetes: what it is, why it is needed, how it works in this lab, and where you meet it at work, and ends with "check yourself" questions. The tested hands-on lessons (seven applications, Docker Compose, fifteen Kubernetes lessons), the twelve troubleshooting labs, the challenges and the guided tutorial are in the repository.

Study material

Everything you need to revise after (or alongside) the hands-on lessons.

File	What it is	Use it when
study-guide.pdf	the 15 concept lessons, the service contract, the troubleshooting method, the capstone, the glossary and the interview questions as one printable PDF (answers expanded, with a table of contents; built by <code>study/tools/build_pdf.py</code>)	you want to read offline, print, or annotate
glossary.md	every term used in the course, A–Z, in one or two plain sentences, with a link to where it is explained	a word in a lesson is unclear
interview-questions.md	25 questions with model answers, from Dockerfiles to troubleshooting	before an interview, or to test yourself
../tutorial/	the guided training course through the repository	you want to be taught, step by step

How to revise

1. Work through the [tutorial](#); it sends you to the lessons, labs and docs in the right order.
2. After each chapter, look up every glossary term you could not explain to a colleague.
3. At the end, answer the interview questions **out loud**, then compare with the model answers. Explaining is a different skill from knowing; it needs practice.
4. Finish with the [capstone](#) and the [knowledge check](#).

Rebuild the PDF

```
pip install markdown
python study/tools/build_pdf.py      (uses a headless Chrome or Edge to print the PDF)
```

01 · The Bookshop architecture

Time: 20 minutes

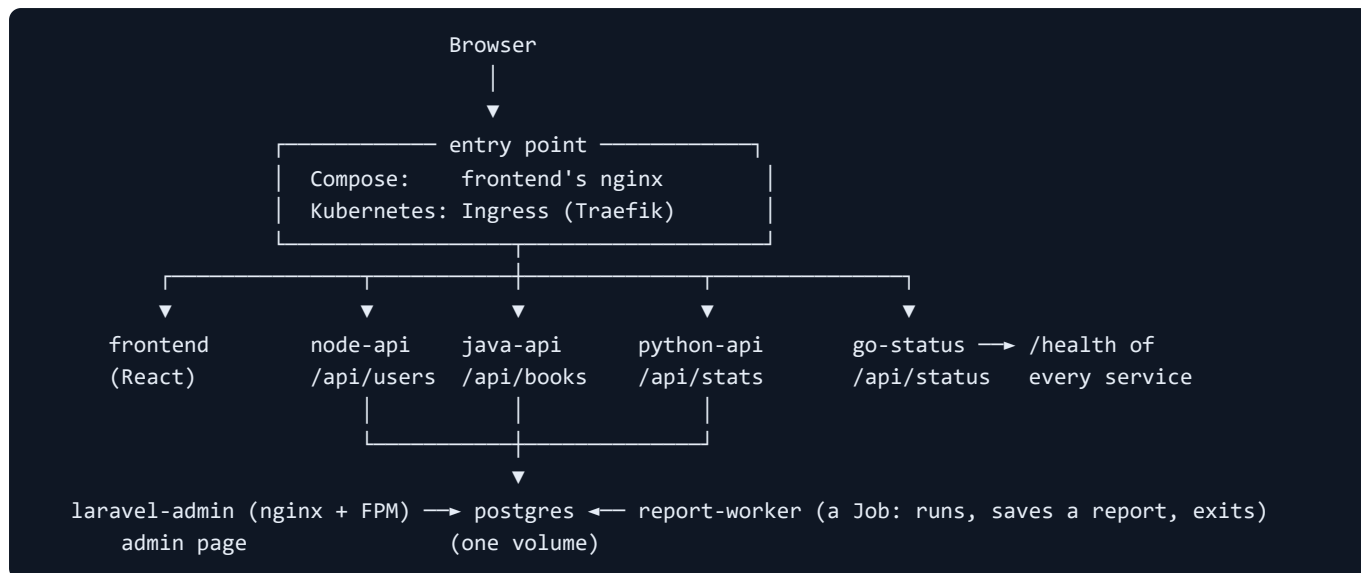
This lab is one small but complete platform, the **Bookshop**. It is made of seven applications written in six languages and frameworks, plus a PostgreSQL database. Every application is deliberately simple: the subject of the lab is **how they are packaged, connected and deployed**, not what they do.

The services

Service	Stack	Port	What it does	Talks to
frontend	React 19 + Vite 8, served by nginx	8080	the web page: books, users, statistics, service status	the APIs, through /api/...
node-api	Node.js 24 + Express 5	3000	users: GET/POST /api/users	PostgreSQL
python-api	Python 3.14 + FastAPI	8000	statistics: GET /api/stats	PostgreSQL (read only)
go-status	Go 1.27, standard library only	8080	status board: GET /api/status checks every service's /health	every service, over HTTP
java-api	Java 25 + Spring Boot 4.1	8080	books: GET /api/books , GET /api/books/{id}	PostgreSQL
laravel-admin	PHP 8.5 + Laravel 13 (nginx + PHP-FPM)	8080	book reviews: list and add (an HTML page)	PostgreSQL
report-worker	JavaScript on Node.js 24	–	a batch job: collects statistics and status, saves one report row, exits	python-api , go-status , PostgreSQL
postgres	PostgreSQL 18	5432	the database bookshop	–

Each service owns its tables: node-api → users , java-api → books , laravel-admin → reviews , report-worker → reports . python-api only reads. The full rules every service follows are in [CONTRACT.md](#).

The picture



One request, from the browser to the database

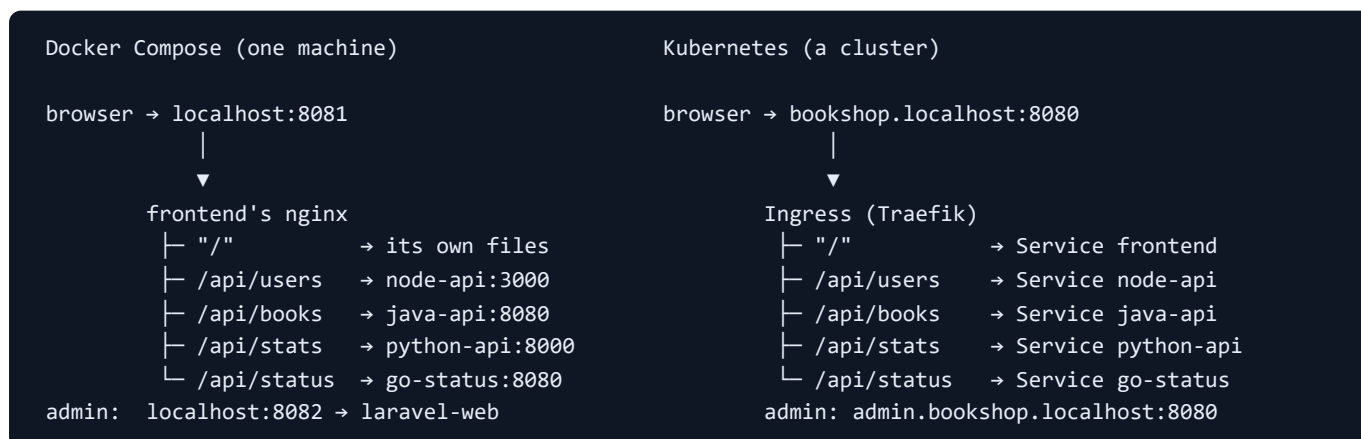
You open the Bookshop page and the Books panel loads. This is what happens in Kubernetes:

- | | | |
|--------------------|--|---|
| 1. Browser Service | GET http://bookshop.localhost:8080/ | → the Ingress sends "/" to the frontend |
| 2. frontend runs) | returns index.html and the JavaScript bundle | (nginx serves static files, nothing else runs) |
| 3. Browser | runs React; React calls GET /api/books | (a relative URL: same host, same port) |
| 4. Ingress | rule "path /api/books → java-api:8080" | (Traefik reads this rule from the Ingress object) |
| 5. Service | java-api picks one Ready java-api Pod | (only Pods that pass their readiness probe) |
| 6. java-api | SELECT id, title, author, year FROM books | (host "postgres", port 5432) |
| 7. Service | postgres → the postgres-0 Pod | |
| 8. back | rows → JSON → Ingress → browser → React draws the list | |

The code that renders the page runs **in your browser**, not in the cluster. The frontend container only hands out files. That is why the API calls go through the same entry point as the page.

The same platform in Compose and in Kubernetes

The applications and images are identical. What changes is **who routes the traffic**:



The frontend's nginx can proxy `/api/...` by itself ([default.conf.template](#)), which is what makes the page work in Compose without anything else. In Kubernetes the [Ingress](#) does that job for every Pod, so the API traffic never passes through the frontend at all.

Why every service keeps its name

`node-api` is called `node-api` in Compose **and** in Kubernetes. That is not a coincidence: both platforms give every service a DNS name.

	Compose	Kubernetes
Who answers the name <code>node-api</code> ?	Docker's embedded DNS on the project network	CoreDNS: the Service <code>node-api</code> in namespace <code>bookshop</code>
What does the name point to?	the container(s) of the service	the Service's virtual IP, which spreads requests over the Ready Pods
Full name	<code>node-api</code>	<code>node-api.bookshop.svc.cluster.local</code> (short <code>node-api</code> works inside the namespace)

Because the names match, the applications need **no change** between Compose and Kubernetes:

`DB_HOST=postgres` works in both, and so does `go-status`'s target list (`node-api=http://node-api:3000/health,...`). Configuration differs; code does not.

Check yourself

▼ Which component runs the React code, the frontend container or the browser?

The browser. The frontend container (nginx) only serves `index.html` and the built JavaScript files. React then runs in the browser and calls the APIs over HTTP.

▼ In Kubernetes, does a request for `/api/books` pass through the frontend?

No. The Ingress has its own rule for `/api/books` and sends it straight to the `java-api` Service. In Compose there is no Ingress, so the frontend's nginx forwards it.

▼ Why does `DB_HOST=postgres` work in both Compose and Kubernetes?

Both give the database a DNS name `postgres` : in Compose the service name, in Kubernetes the Service name in the same namespace. The application just resolves the name.

▼ Which service writes to the `reviews` table, and which only reads it?

`laravel-admin` owns and writes it (its migrations create it). `python-api` reads it to count reviews.

▼ What is special about `report-worker` compared with the other services?

It is not a server. It starts, does one job (collect statistics, save a report) and exits. In Compose it is run on demand; in Kubernetes it becomes a Job, created on a schedule by a CronJob.

02 · Why multiple technology stacks?

Time: 15 minutes

Real companies are never "one language"

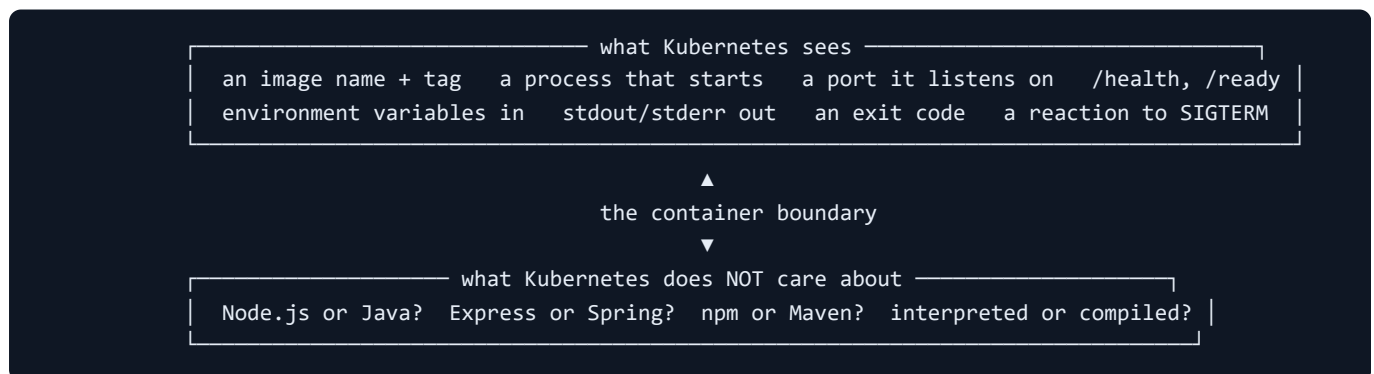
Open the repositories of almost any company older than a few years and you will find several languages side by side:

Why	Example
Teams choose what fits the job	a data team writes Python, a web team writes JavaScript/TypeScript
History	the billing system from 2014 is Java, the new services are Go
Libraries and ecosystems	machine learning in Python, enterprise integrations in Java, a CMS in PHP
Performance or footprint	a small, fast network service in Go; a heavy business application on the JVM
Acquisitions and vendors	a bought product arrives with its own stack

A DevOps or platform engineer rarely gets to choose. The job is to **run all of them the same way**: build, ship, configure, observe and update them with one set of tools. Containers and Kubernetes are how that is done today.

What Kubernetes sees, and what it does not

Kubernetes never looks inside your code. It manages **containers**, and it only sees what a container shows on the outside:



Kubernetes needs	Why
an image it can pull	to start the container on any node
a port	for the Service to send traffic to
health endpoints	to know when to restart (liveness) and when to send traffic (readiness)
configuration from environment variables	so one image runs in every environment (ConfigMaps and Secrets fill them)
logs on stdout/stderr	<code>kubectl logs</code> reads exactly that, nothing else

Kubernetes needs	Why
a meaningful exit code	a Job succeeds with 0 and fails with anything else; a crashing server is restarted
graceful shutdown on SIGTERM	rolling updates and scale-downs send SIGTERM before they stop a container

The contract is the interface

Because Kubernetes only sees the outside, this lab defines that outside once, in [CONTRACT.md](#), and every application follows it, whatever its language:

Rule	Node.js	Python	Go	Java	PHP	JavaScript job
port from <code>PORT</code>	✓	✓	✓	✓	(nginx: 8080, FPM: 9000)	– (no server)
<code>/health</code> (process alive)	✓	✓	✓	✓	✓	–
<code>/ready</code> (database answers)	✓	✓	– (no database)	✓	✓	–
config only from environment	✓	✓	✓	✓	✓	✓
missing <code>DB_PASSWORD</code> → clear message, exit 1	✓	✓	–	✓	✓	✓
logs to stdout	✓	✓	✓	✓	✓ (stderr)	✓
graceful SIGTERM	✓	✓	✓	✓	✓ (FPM)	– (exits on its own)
non-root user in the image	✓	✓	✓	✓	✓	✓

The result: the Kubernetes manifests of `node-api`, `python-api` and `java-api` are almost identical. Only the image, the port, the memory numbers and (for Java) a startup probe differ. Compare [node-api/deployment.yaml](#) with [java-api/deployment.yaml](#).

JavaScript is a language, Node.js is a runtime

Three parts of this lab are written in JavaScript, and they run in three different places:

```

JavaScript (the language)
├── in the BROWSER ..... frontend (React)    the code is built into files; your browser runs it
├── on NODE.JS (a runtime that runs JavaScript outside the browser)
│   ├── a SERVER ..... node-api (Express)    starts, listens on port 3000, runs until stopped
│   └── a BATCH JOB ... report-worker         starts, does one job, exits (no framework at all)

```

- **JavaScript** is the programming language.
- **Node.js** is a runtime: the V8 engine plus file, network and process access, so JavaScript can run outside a browser. It is to JavaScript roughly what the JVM is to Java bytecode.

- **Express** is a framework on top of Node.js for HTTP servers. `report-worker` uses no framework: it is plain JavaScript with one library (the PostgreSQL client).

For Kubernetes the difference that matters is not the language but the **shape of the workload**: `node-api` is a long-running server (a Deployment), `report-worker` runs to completion (a Job, created on a schedule by a CronJob), and the React code never runs in the cluster at all, only the nginx that serves it.

Why not more stacks?

It would be easy to add .NET, Ruby or Rust. They were left out on purpose, because each would repeat a lesson that an existing service already teaches:

Candidate	What it would show	Already shown by
.NET	compiled to an intermediate language, needs a runtime, multi-stage SDK → runtime image	<code>java-api</code> (JDK → JRE)
Rust	one static binary in a tiny final image	<code>go-status</code>
Ruby / Django	interpreted, dependencies from a lock file, a web server process	<code>python-api</code> , <code>node-api</code> , <code>laravel-admin</code>
Next.js / Vue	a JavaScript frontend built into static files	<code>frontend</code>

Seven applications that each teach something different are more useful than ten that repeat each other. Once you have done these, adding any other stack is the same four steps: a Dockerfile, an image, a Compose service, and a Deployment plus Service.

Check yourself

▼ Name four things Kubernetes needs from a container, regardless of its language.

An image to pull, a port to send traffic to, health endpoints for its probes, and configuration through environment variables (also: logs on stdout, meaningful exit codes, graceful SIGTERM handling).

▼ Why are the Deployments of `node-api` and `python-api` almost the same file?

Both follow the same contract: port from `PORT` , `/health` and `/ready` , database settings from the same environment variables. Kubernetes only sees that outside behaviour, so only the image, the port and the resource numbers differ.

▼ Is Node.js a programming language?

No. JavaScript is the language; Node.js is a runtime that executes JavaScript outside the browser.

▼ Which Kubernetes resource fits `report-worker`, and why not a Deployment?

A Job (scheduled by a CronJob). It runs to completion and exits with 0. A Deployment expects a process that keeps running and would restart it forever after every successful exit.

▼ Does the React code of the frontend run inside the Kubernetes cluster?

No. It is built into static files at image build time; the cluster only runs nginx, which serves those files. The browser runs the React code.

03 · Dockerfiles across stacks

Time: 25 minutes

Seven applications, eight Dockerfiles (Laravel has two images). Put them side by side and the same handful of patterns appears in every one, whatever the language.

The eight images at a glance

Sizes are the "disk usage" of `docker images` on the machine that recorded this lab (Docker Desktop 29); the application READMEs show the full output.

Image	Base of the final image	Build stage?	Runs as	Final size	Build stage size
<code>node-api:1.0.0</code>	<code>node:24.21-alpine</code>	no	<code>node</code> (uid 1000)	251 MB	–
<code>report-worker:1.0.0</code>	<code>node:24.21-alpine</code>	no	<code>node</code> (uid 1000)	246 MB	–
<code>python-api:1.0.0</code>	<code>python:3.14-slim</code>	yes: <code>pip install</code> into a venv	uid 10001	261 MB	–
<code>go-status:1.0.0</code>	<code>gcr.io/distroless/static-debian12:nonroot</code>	yes: <code>go build</code>	<code>nonroot</code> (uid 65532)	16.4 MB	381 MB (<code>golang:1.27-alpine</code>)
<code>java-api:1.0.0</code>	<code>eclipse-temurin:25-jre-alpine</code>	yes: Maven builds the JAR	uid 10001	348 MB	677 MB
<code>frontend:1.0.0</code>	<code>nginxinc/nginx-unprivileged:1.30.5-alpine</code>	yes: <code>npm run build</code>	<code>nginx</code> (uid 101)	81.8 MB	334 MB
<code>laravel-fpm:1.0.0</code>	<code>php:8.5-fpm-alpine</code>	yes: <code>composer install</code>	<code>www-data</code> (uid 82)	222 MB	–
<code>laravel-web:1.0.0</code>	<code>nginxinc/nginx-unprivileged:1.30-alpine</code>	no	<code>nginx</code> (uid 101)	81.5 MB	–

Startup time matters for Kubernetes too. Only `java-api`'s was measured in detail (Spring's own log line, see [java-api/README.md](#)): 2.6 s with every CPU of a laptop, 4.6 s limited to one CPU, 9.3 s limited to half a CPU. That number is why only `java-api` has a startup probe (08).

Three families

INTERPRETED: the image contains the runtime + your source code + dependencies
 node-api, report-worker (Node.js) python-api (Python) laravel-fpm (PHP)
 source → [runtime image + dependencies] → the runtime reads the source at start

COMPILED: a build turns source into something else; the final image only needs what runs it
 go-status (Go): source → go build → ONE static binary → [distroless: binary only]
 java-api (Java): source → Maven → app.jar (bytecode) → [JRE + app.jar]

STATIC WEB: a build turns source into files that the BROWSER runs
 frontend (React): source → npm run build → dist/ (HTML, JS, CSS) → [nginx + dist/]

Family	What must be in the final image	What must NOT be
interpreted	the runtime, production dependencies, your code	development dependencies, test tools, caches
compiled	the binary (Go) or the JRE + JAR (Java)	the compiler, Maven, the source code
static web	a web server and the built files	Node.js, <code>node_modules</code> , the source code

Multi-stage builds

A multi-stage Dockerfile has several `FROM` lines. Each stage starts from a clean base; only what you explicitly `COPY --from=` an earlier stage reaches the final image. The Go Dockerfile is the clearest example ([go-status/Dockerfile](#)):

```
FROM golang:1.27-alpine AS build          # 381 MB: the whole Go toolchain
...
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags "-s -w -X main.version=${APP_VERSION}" -o
/out/go-status .

FROM gcr.io/distroless/static-debian12:nonroot # no shell, no package manager, no libc
COPY --from=build /out/go-status /go-status    # ← the only thing that crosses over
USER nonroot:nonroot
ENTRYPOINT ["/go-status"]
```

Which images use it and what they leave behind:

Image	Left behind in the build stage
<code>go-status</code>	the Go compiler and standard library sources
<code>java-api</code>	Maven, the full JDK, the downloaded dependency cache, the source code
<code>frontend</code>	Node.js, <code>node_modules</code> , the React and Vite sources
<code>python-api</code>	nothing large: the venv is copied over (the stage keeps the final image free of build leftovers)
<code>laravel-fpm</code>	Composer and its download cache

`node-api` and `report-worker` have no build step, so one stage is enough: `npm ci --omit=dev` installs only what runs in production.

Why Go's image is 16 MB and Java's is 348 MB

	go-status	java-api
What runs	one statically linked binary (<code>CGO_ENABLED=0</code>)	<code>java -jar app.jar</code>
What it needs at run time	the Linux kernel. Nothing else	a Java runtime (JRE): the JVM, its class library, its native libraries
Final base	distroless "static": certificates, a non-root user, timezone data	<code>eclipse-temurin:25-jre-alpine</code>
Final size	16.4 MB	348 MB

The Go compiler resolves everything at build time, so the result can run on an almost empty image. Java compiles to bytecode, which needs the JVM at run time; the JRE alone is most of the 348 MB. Neither is "better": the JVM buys you a mature ecosystem and runtime optimisation, Go buys you a tiny, fast-starting image. What matters for operations is that both final images contain **only what runs**, not the tools that built them.

Dependency layers and the build cache

Docker reuses a layer from its cache as long as the instruction and its inputs have not changed. Every Dockerfile here copies **the dependency description first**, installs, and only then copies the source code:

Stack	Copied first	Install step	Cached until...
Node.js	<code>package.json</code> , <code>package-lock.json</code>	<code>npm ci --omit=dev</code>	the lock file changes
Python	<code>requirements.txt</code>	<code>pip install -r requirements.txt</code>	the requirements change
Go	<code>go.mod</code>	<code>go mod download</code>	the module file changes
Java	<code>pom.xml</code>	<code>mvn dependency:go-offline</code>	the POM changes
PHP	<code>composer.json</code> , <code>composer.lock</code>	<code>composer install --no-dev</code>	the lock file changes
React	<code>package.json</code> , <code>package-lock.json</code>	<code>npm ci</code>	the lock file changes

Change one line of application code, rebuild, and only the layers after `COPY src ...` run again. The slow part (downloading dependencies) is skipped; you see `CACHED` next to it in the build output. For `java-api`, whose Maven dependency download is the longest step of any image here, this turns a long first build into a short rebuild.

.dockerignore

`docker build` sends a folder (the build context) to the builder. `.dockerignore` keeps things out of it, for speed and for safety:

Image	Ignores, for example
<code>node-api</code> , <code>report-worker</code> , <code>frontend</code>	<code>node_modules</code> (installed inside the build), <code>.env</code>
<code>python-api</code>	<code>__pycache__</code> / , <code>.venv</code> /
<code>java-api</code>	<code>target</code> / (build output), IDE folders

Image	Ignores, for example
laravel-admin	src/vendor , src/.env , src/.env.* (but not .env.example), logs and caches

A local `node_modules` or `vendor` folder would overwrite the clean install inside the image; a local `.env` would put your secrets into it.

Non-root users

Every final image runs as an unprivileged user (the table above). Some base images bring one (`node` , `www-data` , `nginx` in `nginx-unprivileged`, `nonroot` in `distroless`); `python-api` and `java-api` create one with a fixed number (10001). A **numeric** user ID matters in Kubernetes: with `runAsNonRoot: true` the kubelet can only verify a number. That is why the Deployments here also set `runAsUser` explicitly (for example `runAsUser: 1000` for `node-api`, whose image says `USER node`).

Check yourself

▼ Why does the frontend image not contain Node.js?

Node.js is only needed to build the React app into static files. The final stage starts from `nginx` and copies only `dist/` . The browser runs the JavaScript; the container only serves files.

▼ Which line makes the Go binary able to run on an image without a C library?

`CGO_ENABLED=0` in the `go build` command: the binary is statically linked and needs nothing from the base image.

▼ You change one line in `node-api/src/server.js` . Which steps run again on rebuild?

Only the layers from `COPY src ./src` onwards. `npm ci` comes from the cache because `package.json` and `package-lock.json` did not change.

▼ Why is `node_modules` in `node-api's .dockerignore` ?

Dependencies are installed inside the image with `npm ci` ; a local `node_modules` (possibly built for another OS) would be sent to the builder for nothing and could overwrite the clean install.

▼ Why do the Deployments set `runAsUser` even though the images have a `USER` line?

`runAsNonRoot: true` can only be verified for a numeric user. Images like `node-api` say `USER node` (a name), so the Pod spec sets the number (1000) explicitly.

04 · Images, tags and registries

Time: 20 minutes

Anatomy of an image name

```
ghcr.io / sufyanahmadkamboh / bookshop-node-api : 1.0.0 @ sha256:3f2a...
```

registry (where) namespace (whose) repository (what) tag (which version) digest (exactly these bytes, can never change)

Part	Default when you leave it out	Example in this lab
registry	docker.io (Docker Hub)	ghcr.io for the published images; none for local images
namespace	library (Docker's official images)	postgres:18.6-alpine is really docker.io/library/postgres:18.6-alpine
tag	latest	1.0.0
digest	–	kindest/node:v1.37.0@sha256:a1ed56cf... in kind-config.yaml

A **tag** is a label that can be moved to another image at any time. A **digest** is the hash of the image content: the same digest always means the same bytes.

Why never latest

latest is just a tag name, with no meaning of "newest". Using it in a deployment causes three classic problems:

Problem	What happens
Not reproducible	two nodes pull node-api:latest an hour apart and run different code
No visible change	you push a new latest, re-apply the same YAML: nothing changes, because the manifest did not change
No rollback target	"go back to the previous version": which one was it? latest already points somewhere else

So every image in this lab carries its **version as the tag**, and the version is baked into the image too (ARG APP_VERSION=1.0.0 , shown by every service's /health):

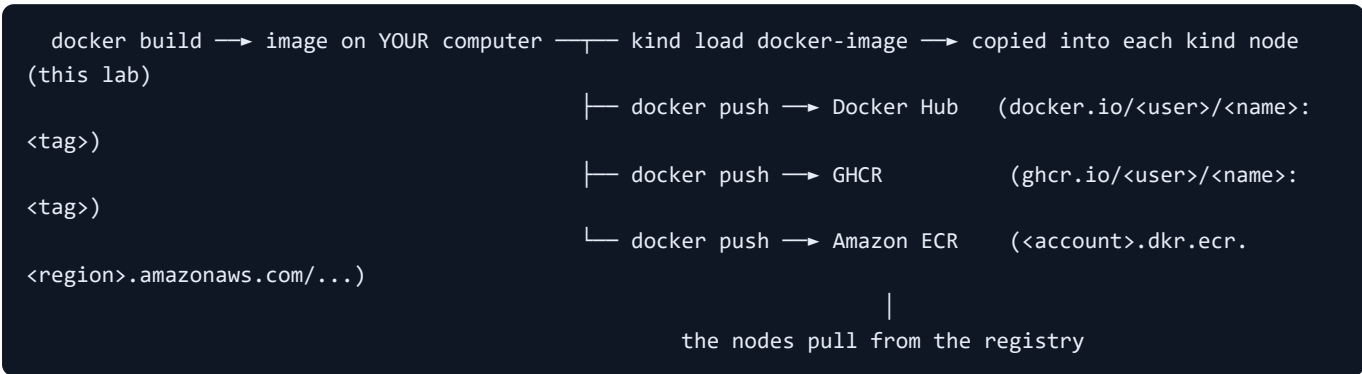
```
node-api:1.0.0    python-api:1.0.0    go-status:1.0.0    java-api:1.0.0
frontend:1.0.0    laravel-fpm:1.0.0    laravel-web:1.0.0    report-worker:1.0.0
```

The rolling-update lesson builds node-api:1.1.0 and changes the Deployment's image from 1.0.0 to 1.1.0 . Because the tag in the manifest changes, Kubernetes sees a new version and rolls it out; kubectl

`rollout undo` goes back to the Pod template with `1.0.0`. Version tags make updates and rollbacks visible and exact. See 12.

How does the image get into the cluster?

Kubernetes never builds images. Each node's container runtime pulls the image named in the Pod spec. It needs a way to get it:



Option	When	What you need
<code>kind load docker-image</code>	local kind clusters (this lab)	nothing: the image is copied straight into the node containers
Docker Hub	public images, small projects	an account; <code>docker login</code> ; private images need an <code>imagePullSecret</code>
GHCR (GitHub Container Registry)	projects on GitHub	the repository's <code>GITHUB_TOKEN</code> in CI can push; public packages need no login to pull
ECR (or ACR, Artifact Registry)	clusters in a cloud	the nodes' cloud identity is allowed to pull

This lab uses `kind load` so you need no account and no internet upload:

```
kind load docker-image node-api:1.0.0 --name bookshop
```

The published copies use the GHCR naming from [CONTRACT.md](#): `ghcr.io/sufyanahmadkamboh/bookshop-<service>:<version>`. Same image, different name: `docker tag` adds a name, `docker push` uploads it.

imagePullPolicy

Every Deployment in this lab says:

```
image: node-api:1.0.0
imagePullPolicy: IfNotPresent # use the image loaded into the cluster (kind load docker-image)
```

Policy	Behaviour	Default when
<code>IfNotPresent</code>	use the node's copy if there is one, pull only if missing	the tag is not <code>latest</code>
<code>Always</code>	ask the registry every time a container starts (the image is downloaded only if the digest differs)	the tag is <code>latest</code> or missing

Policy	Behaviour	Default when
Never	only ever use the node's copy	–

With `kind load` there is no registry that knows `node-api:1.0.0`, so `Always` would fail. That is one of the troubleshooting labs.

ErrImagePull and ImagePullBackOff

When a node cannot get the image, the Pod never starts:

NAME	READY	STATUS	RESTARTS	AGE
node-api-7c9d8b6f5d-x2k4p	0/1	ImagePullBackOff	0	40s

- `ErrImagePull`: the last pull attempt failed.
- `ImagePullBackOff`: Kubernetes waits longer and longer between attempts (back-off).

The reason is in the Pod's events, `kubectl describe pod <name>`: a misspelt name or tag ("not found"), a private image without credentials ("pull access denied" / "unauthorized"), or no network to the registry. The fix is always in the image reference or in the access to the registry, never in the application.

Check yourself

▼ What is the full name of `postgres:18.6-alpine`?

`docker.io/library/postgres:18.6-alpine`: Docker Hub, the official-images namespace `library`, repository `postgres`, tag `18.6-alpine`.

▼ Why does re-applying a Deployment with `image: node-api:latest` after pushing a new latest often change nothing?

The manifest did not change, so Kubernetes sees no new Pod template and starts no rollout. Running Pods keep the image they started with.

▼ How do images get into the kind cluster of this lab?

`kind load docker-image <name>:<tag> --name bookshop` copies the image from your local Docker into every kind node. No registry is involved.

▼ What would `imagePullPolicy: Always` do with a kind-loaded image?

It would try to pull `node-api:1.0.0` from Docker Hub, where it does not exist, and the Pod would end in `ErrImagePull` / `ImagePullBackOff`.

▼ Tag or digest: which one guarantees the exact same image every time?

The digest. A tag can be moved to another image; a digest is the hash of the content.

05 · Docker Compose

Time: 25 minutes

Eight containers started by hand mean eight long `docker run` commands, two networks, a volume and the right order. Docker Compose describes all of it in one file, `compose/docker-compose.yml`, and starts it with one command.

```
Dockerfile → docker build → image
Dockerfile → docker build → image
Dockerfile → docker build → image
volume
... × 8

docker-compose.yml → docker compose up → 8 running containers
(services, networks, on 2 networks, 1
volumes, environment)
```

Services: build or image

Each entry under `services:` becomes one or more containers. A service either **builds** its image or **uses** one:

```
node-api:
  build: { context: ../applications/node-api, args: { APP_VERSION: "1.0.0" } } # how to build it
  image: node-api:1.0.0 # the name to give it
```

With both `build:` and `image:`, Compose builds the image and tags it `node-api:1.0.0`: the very same name the Kubernetes manifests use. `postgres` only has `image: postgres:18.6-alpine`: it is downloaded, never built. `laravel-fpm` has no `build:` because `laravel-migrate` already builds that same image.

Networks: who may talk to whom

frontend network

```
frontend  laravel-web
go-status
```

backend network

```
frontend  laravel-web  node-api  python-api
java-api  go-status    laravel-fpm  postgres
laravel-migrate  report-worker
```

`postgres` is only on `backend`. A container that is only on `frontend` could not even resolve the name `postgres`. Containers that must bridge the two (the frontend proxies to the APIs, `go-status` checks everyone) are on both.

Service discovery: names, not IP addresses

On a Compose network every service is reachable by its **service name**: Docker's embedded DNS (127.0.0.11) answers `postgres`, `node-api`, `laravel-fpm` with the containers' current addresses. That is why the configuration says `DB_HOST: postgres` and never an IP address. Containers get new addresses when they are recreated; names stay.

Environment and the .env file

The database settings are written once and reused with a YAML anchor:

```
x-db-env: &db-env
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: bookshop
  DB_USER: bookshop
  DB_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD in compose/.env}
```

- `${DB_PASSWORD}` is read from the shell or from `compose/.env` (not committed; `.env.example` shows the shape).
- `?:message` makes the variable **required**: `docker compose up` stops with that message instead of starting a database with an empty password.
- `environment: *db-env` gives a service the whole block; `<<: *db-env` merges it and lets you add keys (the Laravel services add `APP_KEY`).

Volumes: data that outlives containers

```
volumes:
  - pgdata:/var/lib/postgresql
...
volumes:
  pgdata: {}
```

`pgdata` is a named volume managed by Docker. `docker compose down` removes the containers but keeps the volume; only `docker compose down -v` deletes it, and the data with it.

Ports: from your computer into a container

```
ports: ["127.0.0.1:8081:8080"] # frontend: host 127.0.0.1:8081 → container port 8080
```

`HOST_IP:HOST_PORT:CONTAINER_PORT`. Binding to `127.0.0.1` keeps the port off your network: only your own computer can reach it. Only two services publish ports, `frontend` (8081) and `laravel-web` (8082). The APIs and the database are reachable **only** from other containers: nothing outside needs them directly.

Start order: depends_on and healthchecks

`depends_on` alone only waits until a container has **started**, not until the program inside is ready. A Spring Boot API that has started may still need seconds before it answers. The lab therefore combines `depends_on` with **conditions**:

Condition	Meaning	Used for
<code>service_started</code>	the container is running	<code>go-status</code> (its distroless image has no shell or <code>wget</code> to run a check)
<code>service_healthy</code>	the container's <code>healthcheck</code> passes	<code>postgres</code> before the APIs; the APIs before <code>frontend</code>

Condition	Meaning	Used for
<code>service_completed_successfully</code>	a one-off container exited with 0	<code>laravel-migrate</code> before <code>laravel-fpm</code>

The healthchecks call each API's `/ready` endpoint, for example:

```
healthcheck:                                # Compose's version of a readiness check (Kubernetes:
readinessProbe)
  test: ["CMD", "wget", "-q0-", "http://127.0.0.1:8080/ready"]
  interval: 3s
  timeout: 3s
  retries: 40
```

One-off tasks and profiles

- `laravel-migrate` runs `php artisan migrate --force --seed` once and exits (`restart: "no"`). The real FPM service starts only after it succeeded. One image, two roles: the same `laravel-fpm:1.0.0` with a different command.
- `report-worker` has `profiles: [jobs]`. `docker compose up` ignores it; you run it on demand with `docker compose run --rm report-worker`, and it exits after saving its report.

The commands

Command	Does
<code>docker compose build</code>	builds every service that has <code>build:</code>
<code>docker compose up -d</code>	creates networks and volumes, starts everything in dependency order, in the background
<code>docker compose ps</code>	the containers of this project, their state and health
<code>docker compose logs -f node-api</code>	the stdout/stderr of a service (follow with <code>-f</code>)
<code>docker compose exec node-api sh</code>	a shell (or any command) inside a running container
<code>docker compose run --rm report-worker</code>	a new one-off container of a service, removed afterwards
<code>docker compose down</code>	stops and removes containers and networks (volumes stay)
<code>docker compose down -v</code>	... and the volumes: the database is gone

What Compose is for, and where it stops

Compose is excellent for **one machine**: development, demos, tests in CI, small single-server deployments. It does not:

- spread containers over several machines, or move them when a machine dies;
- replace a crashed container on another host (it can only restart it in place with a `restart:` policy);
- update a service without downtime (no rolling updates with readiness gating);
- give you per-service traffic routing from outside without an extra proxy;

- scale based on load.

Those are exactly the things Kubernetes adds. The next lesson maps every part of this file to a Kubernetes resource.

Check yourself

▼ What is the difference between `depends_on: [x]` and `depends_on: {x: {condition: service_healthy}}`?

The first waits only until container x has started. The second waits until x's healthcheck passes, i.e. until the application inside actually answers.

▼ Why can no container on only the `frontend` network connect to PostgreSQL?

`postgres` is only on the `backend` network. Containers can only reach (and resolve the names of) containers on a network they share.

▼ What does `${DB_PASSWORD:?set DB_PASSWORD in compose/.env}` do when the variable is missing?

Compose refuses to start and prints the message. Nothing starts with an empty password.

▼ You run `docker compose down`. Are the reviews and users still there after `up -d`?

Yes. `down` keeps named volumes; the data lives in the `pgdata` volume. Only `down -v` deletes it.

▼ Why is report-worker in a profile?

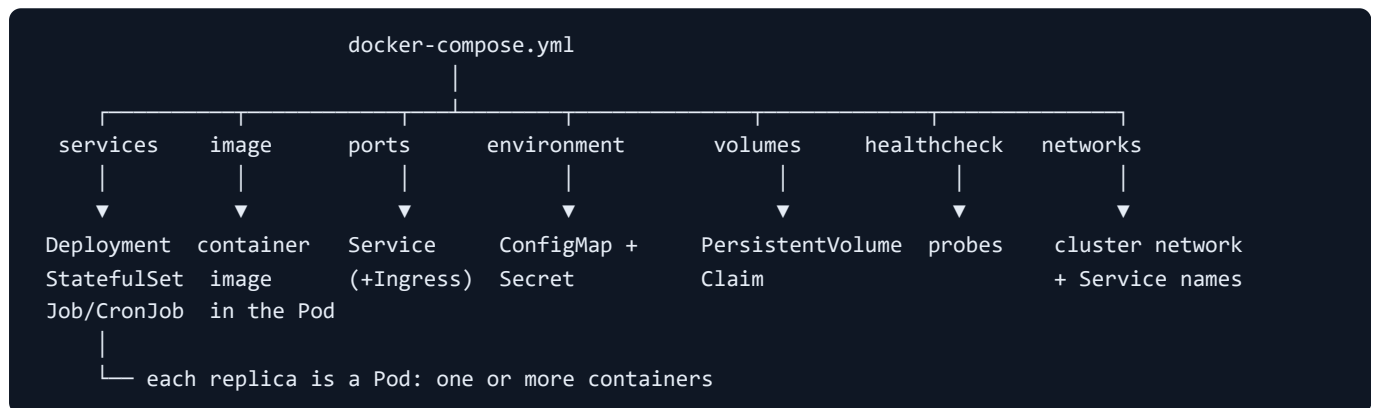
It is a batch job, not a server. With the `jobs` profile, `up` does not start it; you run it when you want a report with `docker compose run --rm report-worker`.

06 · From Docker Compose to Kubernetes

Time: 40 minutes. The most important lesson of the lab.

Compose and Kubernetes solve related problems. Compose answers "how do I run these containers together on **this machine**?". Kubernetes answers "how do I keep these applications running on **a cluster of machines**, update them without downtime and recover from failures by itself?". The images are the same. The description is not.

Kubernetes does **not** mean "take docker-compose.yml and magically run it". Every line of the Compose file expresses a requirement of the application; in Kubernetes each requirement becomes the resource made for it.



The mapping table

Docker Compose	Kubernetes	In this lab
a service (long-running)	Deployment (or StatefulSet when it has its own data)	node-api ... → Deployments; postgres → StatefulSet
a service that runs once	Job / CronJob	laravel-migrate → Job; report-worker → CronJob
a container	a Pod (one or more containers)	laravel-web + laravel-fpm → one Pod with two containers
image:	containers[].image	identical names: node-api:1.0.0
ports:	a Service (inside the cluster), an Ingress (from outside)	Services for every API; one Ingress
environment:	ConfigMap (plain values) and Secret (sensitive values)	bookshop-config, db-credentials, laravel-app-key
volumes: (named)	PersistentVolumeClaim	postgres' volumeClaimTemplates
networks:	one flat cluster network + Services for names (+ NetworkPolicies to restrict)	every Pod can reach every Service by name
deploy.replicas / --scale	spec.replicas	node-api: 2, python-api: 2, frontend: 2

Docker Compose	Kubernetes	In this lab
healthcheck:	probes: readiness, liveness, startup	/ready → readinessProbe, /health → livenessProbe
depends_on:	no ordering: readiness probes + applications that retry	the APIs start before the database answers and become Ready later
restart:	built into the workload type (restartPolicy)	Deployments always restart; Jobs Never / OnFailure

Each row, with the real files of the lab side by side:

service → Deployment

```
# compose/docker-compose.yml
node-api:
  image: node-api:1.0.0

as replicas says

# kubernetes/node-api/deployment.yaml
kind: Deployment
metadata: { name: node-api, namespace: bookshop }
spec:
  replicas: 2
  selector:
    matchLabels: { app: node-api }
  template:
    # ← the Pod to create, as often

    metadata:
      labels: { app: node-api, ... }
    spec:
      containers:
        - name: node-api
          image: node-api:1.0.0
```

A Deployment is not a container; it is a **promise**: "keep 2 Pods of this template running". If a Pod dies or a node disappears, the Deployment's controller creates a replacement. Compose has no such controller across machines.

service with data → StatefulSet

```
# compose
postgres:
  image: postgres:18.6-alpine
  volumes:
    - pgdata:/var/lib/postgresql

across restarts

# kubernetes/database/postgres.yaml
kind: StatefulSet
spec:
  serviceName: postgres
  replicas: 1
  ...
  volumeClaimTemplates:
    # one disk per Pod, kept

    - metadata: { name: data }
      spec: { resources: { requests: { storage: 1Gi } } }
```

A StatefulSet gives each Pod a stable name (postgres-0) and its own PersistentVolumeClaim that follows it. A Deployment's Pods are interchangeable and get random names: fine for stateless APIs, wrong for a database.

one-off service → Job, batch service → CronJob

```
# compose
laravel-migrate:
  image: laravel-fpm:1.0.0
  command: ["php","artisan","migrate",
    "--force","--seed"]
  restart: "no"

seed"]

# compose
report-worker:
  profiles: [jobs] # run by hand

# kubernetes/laravel-admin/migrate-job.yaml
kind: Job
spec:
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
        - image: laravel-fpm:1.0.0
          args: ["php","artisan","migrate","--force","--

# kubernetes/report-worker/cronjob.yaml
kind: CronJob
spec:
  schedule: "*/10 * * * *"
  concurrencyPolicy: Forbid
```

A Job runs Pods **until one succeeds** (exit code 0). A CronJob creates such a Job on a schedule. See [14](#).

container → Pod

Most Pods here have one container. `laravel-admin` has two, because nginx and PHP-FPM are one unit: they always run, scale and get replaced together, and they talk over `127.0.0.1` because containers in a Pod share one network namespace:

```
# compose: two services on a network
laravel-fpm:
  image: laravel-fpm:1.0.0
laravel-web:
  image: laravel-web:1.0.0
  environment:
    FPM_HOST: laravel-fpm

# kubernetes/laravel-admin/deployment.yaml: ONE Pod
containers:
  - name: laravel-web
    image: laravel-web:1.0.0
    env: [{ name: FPM_HOST, value: "127.0.0.1" }]
  - name: laravel-fpm
    image: laravel-fpm:1.0.0
```

The only change is one environment variable: `FPM_HOST` points to a service name in Compose and to `127.0.0.1` in the Pod. See [15](#).

ports → Service (+ Ingress)

```
# compose
node-api:
  # no ports: only reachable by name
  # from other containers

this label
frontend:
  ports: ["127.0.0.1:8081:8080"]

# kubernetes/node-api/service.yaml
kind: Service
spec:
  type: ClusterIP
  selector: { app: node-api } # all Ready Pods with
  ports:
    - port: 3000
      targetPort: http
```

A **Service** gives a set of Pods one stable name and virtual IP inside the cluster, and spreads requests over the Ready ones. It replaces both Compose's DNS name and its load spreading. **Publishing a port to the outside** has no one-to-one equivalent: the lab uses one **Ingress** (`ingress.yaml`) that routes `bookshop.localhost` paths to the Services, instead of one host port per service. See [11](#).

environment → ConfigMap + Secret

```
# compose
x-db-env: &db-env
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: bookshop
  DB_USER: bookshop
  DB_PASSWORD: ${DB_PASSWORD:?...}
Git

# kubernetes/config/bookshop-config.yaml (ConfigMap)
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: bookshop
  DB_USER: bookshop
  # the password: a Secret, created with kubectl, never in
  # in the Deployment:
  - name: DB_PASSWORD
    valueFrom: { secretKeyRef: { name: db-credentials,
key: DB_PASSWORD } }
```

Same variable names, same values; they just come from two Kubernetes objects. See [07](#).

healthcheck → probes

```
# compose
java-api:
s
  healthcheck:
    test: ["CMD", "wget", "-qO-",
      "http://127.0.0.1:8080/ready"]
    interval: 3s
process hangs
  retries: 40
DB answers

# kubernetes/java-api/deployment.yaml
startupProbe: # give the JVM up to 60
  httpGet: { path: /health, port: http }
  periodSeconds: 2
  failureThreshold: 30
livenessProbe: # restart if the
  httpGet: { path: /health, port: http }
readinessProbe: # traffic only when the
  httpGet: { path: /ready, port: http }
```

A Compose healthcheck is one check that mostly gates `depends_on`. Kubernetes splits it into three questions and acts on each one: restart, stop sending traffic, or wait. See [08](#).

depends_on → readiness and retrying applications

Kubernetes starts all Pods at the same time; there is no "start node-api after postgres". The lab handles that the way production systems do:

1. the APIs **start without the database** (they do not crash when it is missing),
2. their **readinessProbe** (`/ready`) fails until the database answers, so the Service sends them no traffic,
3. the moment PostgreSQL is up, `/ready` passes and traffic flows.

Ordering by readiness is more robust than ordering by start: it also covers a database that restarts **later**, which `depends_on` never could.

networks → one flat network + Services

Every Pod in the cluster gets its own IP and can reach every other Pod and Service (the CNI plugin makes that work across nodes). There is no `frontend` / `backend` split by default; isolation is added with **NetworkPolicies** when needed. Names come from Services (`node-api` , `postgres`), resolved by CoreDNS.

What does not translate

Compose feature	In Kubernetes
<code>build:</code>	nothing: Kubernetes only runs images. You build (and load or push) them first
<code>depends_on</code> ordering	none: readiness probes + applications that retry
<code>ports: "8081:8080"</code> on your computer	a Service (inside), an Ingress or a NodePort/LoadBalancer (outside)
<code>.env</code> file substitution	ConfigMaps and Secrets, or a templating tool (Helm, Kustomize)
<code>profiles</code>	separate manifests you apply when you want them (or a CronJob instead of "run on demand")

Why not just convert the file?

Tools such as **kompose** read a `docker-compose.yml` and print Kubernetes YAML. They are a fine starting point, but the output still has to be understood and fixed: kompose cannot know that `postgres` needs a StatefulSet, that `laravel-migrate` is a Job, that `laravel-web` and `laravel-fpm` belong in one Pod, which values are secrets, or which endpoint means "ready". Those are decisions about the application, and they are exactly what this lesson is about.

Check yourself

▼ node-api publishes no port in Compose. How do other Pods reach it in Kubernetes?

Through the `node-api` Service: a stable name and virtual IP in front of the Ready `node-api` Pods. In Compose, other containers reach it by its service name on the shared network; publishing a host port was never needed.

▼ Why is postgres a StatefulSet and node-api a Deployment?

`postgres` has data that must stay with it across restarts: a StatefulSet gives it a stable name and its own PersistentVolumeClaim. `node-api` keeps no state; its Pods are interchangeable, which is what a Deployment manages.

▼ What replaces `depends_on: postgres: condition: service_healthy` ?

Nothing orders the start. The APIs start anyway, their readinessProbe (`/ready`) fails until the database answers, and the Service only sends them traffic once they are Ready.

▼ Why are laravel-web and laravel-fpm two services in Compose but one Pod in Kubernetes?

They form one unit that always runs and scales together. In a Pod they share the network namespace and talk over 127.0.0.1; only `FPM_HOST` changes. (Two separate Deployments would also work; the Pod keeps the pair together.)

▼ Which Compose key has no equivalent at all in a Kubernetes manifest?

`build:` . Kubernetes never builds images; they must exist in the cluster or a registry before a Pod can use them.

07 · ConfigMaps and Secrets

Time: 20 minutes

Every application in this lab reads its configuration from environment variables (rule 5 of the [contract](#)). That is what lets **one image** run on a laptop, in Compose and in Kubernetes. Kubernetes fills those variables from two kinds of objects.

	ConfigMap	Secret
For	non-sensitive settings	passwords, keys, tokens
In this lab	<code>bookshop-config</code>	<code>db-credentials</code> , <code>laravel-app-key</code>
Stored in Git?	yes (<code>bookshop-config.yaml</code>)	never : created with <code>kubectl</code> , only an example is committed
Values in the object	plain text	base64-encoded
Access control	normal	can be restricted separately (RBAC), not shown in <code>kubectl describe</code>

The ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: bookshop-config
  namespace: bookshop
data:
  DB_HOST: postgres
  DB_PORT: "5432"           # values are always strings: quote numbers
  DB_NAME: bookshop
  DB_USER: bookshop
  STATS_URL: http://python-api:8000/api/stats
  STATUS_URL: http://go-status:8080/api/status
  ADMIN_URL: http://admin.bookshop.localhost:8080
  TARGETS: >-
    frontend=http://frontend:8080/health,node-api=http://node-api:3000/health,...
```

One object holds the settings several services share. Change the database name here, and every service that reads `DB_NAME` picks it up (after a restart, see below).

The Secrets

They are created from **generated** values, so no password ever exists in a file:

```
kubectl -n bookshop create secret generic db-credentials --from-literal=DB_PASSWORD="$(openssl rand -hex 16)"
kubectl -n bookshop create secret generic laravel-app-key --from-literal=APP_KEY="base64:$(openssl rand -base64 32)"
```

The same password is used by PostgreSQL (`POSTGRES_PASSWORD`) and by every client (`DB_PASSWORD`): both read the key `DB_PASSWORD` of the same Secret, so they can never get out of sync.

Getting values into the container

One variable at a time: `valueFrom`

What every Deployment in this lab does:

```
env:
  - name: DB_HOST
    valueFrom: { configMapKeyRef: { name: bookshop-config, key: DB_HOST } }
  - name: DB_PASSWORD
    valueFrom: { secretKeyRef: { name: db-credentials, key: DB_PASSWORD } }
```

the ConfigMap

Explicit: you see exactly which variables a container gets, and a typo in a key name stops the Pod with a clear error (`CreateContainerConfigError: couldn't find key ...`).

Everything at once: `envFrom`

```
envFrom:
  - configMapRef: { name: bookshop-config }
```

Shorter, but every key becomes a variable, including ones the application does not need (`TARGETS` in `node-api`), and you no longer see in the Deployment what the container depends on. The lab uses `valueFrom` for clarity.

Both can also be **mounted as files** (a volume of type `configMap` or `secret`): useful for whole configuration files such as an nginx config.

base64 is not encryption

```
kubectl -n bookshop get secret db-credentials -o jsonpath='{.data.DB_PASSWORD}' → <the value in base64>
kubectl -n bookshop get secret db-credentials -o jsonpath='{.data.DB_PASSWORD}' | base64 -d → <the password itself>
```

base64 is an **encoding** (so binary values fit into YAML), not a protection. Anyone who can read the Secret object can read the value. What Secrets do give you:

Protection	How
Not in the image	the value is injected at run time; <code>docker history</code> shows nothing
Not in Git	the lab creates Secrets with <code>kubectl create secret</code> ; only an example with a fake value is committed
Separate permissions	RBAC can allow a team to read ConfigMaps but not Secrets
Not printed by default	<code>kubectl describe secret</code> shows sizes, not values

Protection	How
Encryption at rest	when the cluster is configured for it (managed clusters usually offer it), etcd stores Secrets encrypted

For production, teams often keep the real values in an external secret manager (AWS Secrets Manager, HashiCorp Vault, ...) and sync them into Kubernetes, or encrypt them for Git (Sealed Secrets, SOPS). The rule never changes: **never commit a real secret, not even base64-encoded.**

Changing configuration

Environment variables are read **once, when the container starts**. Editing the ConfigMap or Secret does not change running containers:

```
kubectl -n bookshop edit configmap bookshop-config      # change a value
kubectl -n bookshop rollout restart deployment node-api # new Pods start with the new value
```

`rollout restart` replaces the Pods one by one (a rolling update), so the service stays available. Values mounted as **files** are updated in the running container after a short delay, but the application must re-read the file to notice.

Check yourself

▼ Why is `DB_PASSWORD` not in the ConfigMap, although it is "just another setting"?

It is sensitive. Secrets can be protected separately (RBAC, encryption at rest) and are never committed to Git; ConfigMaps are plain configuration meant to be versioned.

▼ Someone says "our password is safe, it is base64-encoded in the Secret". Is it?

No. `base64 -d` reverses it instantly. Protection comes from who may read the Secret (RBAC), from encryption at rest, and from keeping the value out of Git and images.

▼ You change `DB_NAME` in the ConfigMap. Why do the running `node-api` Pods still use the old one?

Environment variables are set when the container starts. Run `kubectl rollout restart deployment node-api` to start new Pods with the new value.

▼ What happens when a Deployment references a Secret key that does not exist?

The container cannot be created: the Pod shows `CreateContainerConfigError`, and `kubectl describe pod` names the missing key.

▼ `env + valueFrom` or `envFrom`: what is the trade-off?

`valueFrom` is explicit (you see each variable and its source) but longer; `envFrom` imports every key of the object in one line but hides what the container really depends on.

08 · Health probes

Time: 25 minutes

The kubelet on each node asks every container, again and again, three different questions. Each question is a **probe**, and each one leads to a different action.

Probe	The question	When it fails	Endpoint in this lab
livenessProbe	"Is this process still alive, or stuck?"	the container is restarted	<code>/health</code>
readinessProbe	"Should this Pod receive traffic right now ?"	the Pod is removed from the Service (no restart)	<code>/ready</code> (services with a database) or <code>/health</code>
startupProbe	"Has it finished starting?"	after the limit: restarted; until then liveness and readiness wait	<code>/health</code> (only <code>java-api</code>)

Liveness answers "Should Kubernetes restart this container?" Readiness answers "Should this Pod receive traffic?"

`/health` and `/ready`: two endpoints on purpose

The [contract](#) gives every API two endpoints:

```
/health → 200 as soon as the process can answer HTTP.      NEVER checks the database. →
livenessProbe
/ready → 200 when the database answers, 503 when it does not. →
readinessProbe
```

Why must `/health` not check the database? Imagine it did, and PostgreSQL restarts for 30 seconds:

<u><code>/health</code> checks the DB (wrong)</u>	<u><code>/health</code> ignores the DB (this lab)</u>
DB down → every API's liveness fails → Kubernetes restarts ALL API containers → restarts do not fix the database → the APIs come back slowly, in a restart loop state	DB down → <code>/ready</code> fails → Pods leave the Services → <code>/health</code> still passes → nothing is restarted DB back → <code>/ready</code> passes → traffic resumes at once No restarts, no cold starts, no lost in-memory

A restart only helps when the **process itself** is broken. A missing dependency is a readiness problem.

The probes in the lab

From `node-api/deployment.yaml` (python-api is the same):

```

livenessProbe:                # "should Kubernetes restart this container?"
  httpGet: { path: /health, port: http }
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:                # "should this Pod receive traffic right now?"
  httpGet: { path: /ready, port: http }
  periodSeconds: 5
  failureThreshold: 2

```

- liveness: checked every 10 s; 3 failures in a row (about 30 s) → restart.
- readiness: checked every 5 s; 2 failures (about 10 s) → out of the Service until it passes again.
- `port: http` refers to the **named** container port, so the number lives in one place.

The other services:

Service	liveness	readiness	Why
go-status , frontend	/health	/health	no database: ready as soon as they run
java-api	/health	/ready	plus a startupProbe (below)
laravel-web (nginx)	/health	/ready	nginx passes both to Laravel
laravel-fpm	tcpSocket: 9000	–	FPM speaks FastCGI, not HTTP: the probe only checks that it accepts connections
postgres	exec: pg_isready	exec: pg_isready	a command inside the container instead of an HTTP call

The startup probe for Java

`java-api` measured its own startup (Spring's log line `Started BookshopApplication in ... seconds`, see [java-api/README.md](#)):

CPU available	Startup
all CPUs of a laptop	2.6 s
1 CPU (the Pod's limit in this lab)	4.6 s
0.5 CPU	9.3 s

A liveness probe that starts checking right away could restart a JVM that is simply still starting, on a busy node again and again. The startup probe holds the other probes back until the application has started once:

```

  startupProbe:                # the JVM needs several seconds: give it up to 60 s before
liveness starts
  httpGet: { path: /health, port: http }
  periodSeconds: 2
  failureThreshold: 30

```

$2\text{ s} \times 30\text{ attempts} = \text{up to } 60\text{ s to start}$, while the liveness probe can stay strict ($3 \times 10\text{ s}$) for the rest of the container's life. Node.js, Python and Go start fast enough not to need one.

What a failing probe looks like

Readiness failing looks like this (an example; the troubleshooting labs record the real thing):

NAME	READY	STATUS	RESTARTS	AGE
node-api-6f678b5b5-kxsn1	0/1	Running	0	2m

`kubectl get endpoints node-api` no longer lists that Pod's IP. `kubectl describe pod` shows events such as `Readiness probe failed: HTTP probe failed with statuscode: 503`.

Liveness failing looks like this (example):

NAME	READY	STATUS	RESTARTS	AGE
node-api-6f678b5b5-kxsn1	1/1	Running	3 (20s ago)	5m

`RESTARTS` goes up; `kubectl logs --previous` shows the output of the container before the restart.

Common mistakes

Mistake	Consequence
liveness checks a dependency (database, another API)	one outage restarts everything that depends on it
no readiness probe	traffic reaches Pods that are still starting: errors during every rollout
liveness without a startup probe on a slow starter	restart loop on slow nodes or with low CPU limits
probing the wrong port or path	the Pod never becomes Ready (readiness) or restarts forever (liveness)
timeouts shorter than the endpoint needs under load	healthy Pods are taken out or restarted when they are busy
readiness and liveness identical and heavy	double work, and a slow dependency turns into restarts

Check yourself

▼ The database is down. Should node-api's containers be restarted?

No. Restarting does not fix the database. `/ready` fails, the Pods leave the Service; `/health` keeps passing, so nothing restarts. When the database is back, they become Ready again on their own.

▼ What does Kubernetes do when a readiness probe fails? And a liveness probe?

Readiness: removes the Pod from the Service endpoints (no traffic), no restart. Liveness: kills and restarts the container.

▼ Why does only java-api have a startupProbe?

The JVM and Spring need seconds to start (4.6 s with one CPU, 9.3 s with half a CPU in the lab's measurements). The startup probe gives it up to 60 s before the liveness probe may restart it. The other

services start fast.

▼ Why is laravel-fpm's liveness probe a `tcpSocket` check?

PHP-FPM speaks FastCGI, not HTTP, so an `httpGet` probe cannot talk to it. A TCP check verifies that it accepts connections on port 9000; the HTTP checks go through nginx (`laravel-web`).

▼ A Pod shows `0/1 Running` with 0 restarts. Which probe is failing?

The readiness probe. The container runs (no restarts, so liveness passes) but is not Ready, so it receives no traffic.

09 · Resources: requests and limits

Time: 20 minutes

Every container in this lab declares how much CPU and memory it needs. Kubernetes uses these numbers twice: once to **decide where a Pod runs**, and again to **stop a container from taking more than its share**.

```
resources:
  requests: { cpu: 50m, memory: 64Mi }    ← "reserve at least this much for me"    (used by the
scheduler)
  limits:  { cpu: "500m", memory: 256Mi } ← "never let me use more than this"    (enforced by the
kernel)
```

Units

Unit	Meaning
1 CPU	one full CPU core (a vCPU in the cloud, a hyperthread on a laptop)
100m	100 milli cores = 0.1 of a core. 250m = a quarter of a core
64Mi	64 mebibytes (64 × 1024 × 1024 bytes). Mi / Gi are binary units; M / G are decimal

What this lab asks for

Taken from the manifests in [kubernetes/](#):

Container	Requests (CPU / memory)	Limits (CPU / memory)	Why this size
frontend (nginx)	10m / 32Mi	200m / 128Mi	serves static files: almost no work
node-api	50m / 64Mi	500m / 256Mi	a small Node.js process
python-api	50m / 96Mi	500m / 256Mi	Python + FastAPI + uvicorn need a bit more memory at start
java-api	250m / 256Mi	1 / 512Mi	the JVM: more memory, and CPU to start quickly
go-status	10m / 16Mi	200m / 64Mi	one small compiled binary
laravel-web (nginx)	10m / 32Mi	200m / 128Mi	static files + FastCGI forwarding
laravel-fpm	50m / 64Mi	500m / 256Mi	PHP-FPM worker processes
report-worker	10m / 32Mi	200m / 128Mi	runs for a few seconds, then exits
postgres	100m / 128Mi	1 / 512Mi	the database: room for caches

Requests: the scheduler's budget

The scheduler places a Pod on a node only if the node's **allocatable** resources minus the **requests** of the Pods already there are at least the new Pod's requests. It does not look at how much the Pods *really* use.

```
node: 4 CPU allocatable
├─ Pods already placed: requests add up to 3.8 CPU (even if they only use 0.5 CPU right now)
└─ new Pod requests 250m → 3.8 + 0.25 > 4 → does not fit → Pending: "Insufficient cpu"
```

So requests that are too **high** waste capacity (Pods stay `Pending` although the node is idle), and requests that are too **low** pack too many Pods onto a node, which then fight for CPU and memory.

Limits: what happens at the ceiling

CPU and memory behave very differently at their limit:

Resource	At the limit	What you see
CPU	the container is throttled : it gets no more CPU time in that period	it runs slower (higher latency); it is not killed
Memory	the kernel's OOM killer ends the process	the container restarts; <code>kubectl describe pod</code> shows <code>Last State: Terminated, Reason: OOMKilled, Exit Code: 137</code>

Exit code 137 is 128 + 9: the process was ended with signal 9 (SIGKILL). The application gets no chance to clean up. A container that keeps hitting its memory limit ends up in `CrashLoopBackOff`.

Quality of Service (QoS) classes

From requests and limits, Kubernetes gives every Pod a QoS class. When a **node** runs out of memory, the kubelet evicts Pods in this order: BestEffort first, Guaranteed last.

QoS class	Rule	In this lab
Guaranteed	every container: requests = limits, for CPU and memory	none
Burstable	at least one request or limit set, but not Guaranteed	every Pod here
BestEffort	no requests and no limits at all	none (and it should stay that way)

Every Pod in the lab is **Burstable**: it reserves a small amount and may use more when the node has room. That fits a laptop. Production services often set memory request = memory limit to avoid surprises.

Java: the heap and the container's memory limit

A JVM decides its maximum heap size when it starts. The `java-api` image starts it with:

```
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Modern JVMs read the **container's** memory limit (512Mi here) instead of the host's memory. With `MaxRAMPercentage=75`, the heap may grow to about 75 % of 512Mi; the rest is left for the JVM's own memory (threads, classes, the garbage collector). Raise the limit and the heap grows with it; set the percentage too high and the process can exceed the limit and be OOMKilled.

CPU matters for the JVM as well. The [java-api lesson](#) measured the same image with different CPU limits:

```
cpus=0.5: Started BookshopApplication in 9.302 seconds
cpus=1: Started BookshopApplication in 4.607 seconds
```

That is why `java-api` has a `startupProbe` ([docs/08](#)) and the largest CPU limit in the lab.

Sizing for a local kind cluster

The kind cluster in this lab is two Docker containers on your computer. Its "nodes" see your computer's CPUs and memory, so the requests above fit easily on a normal laptop. Keep the requests small: three replicas of every service should still fit. Measure before you change numbers:

```
kubectl top pods -n bookshop      (needs the metrics-server add-on, which kind does not install by
default)
kubectl describe node <node>      → "Allocated resources": the sum of requests and limits on that
node
```

Check yourself

▼ A Pod is Pending with "Insufficient memory", but the node uses only 30 % of its memory. How?

The scheduler counts **requests**, not real usage. The requests already placed on the node, plus this Pod's request, exceed the node's allocatable memory.

▼ What happens when a container reaches its CPU limit? And its memory limit?

CPU: it is throttled and runs slower, but keeps running. Memory: the process is killed (OOMKilled, exit code 137) and the container restarts.

▼ Which QoS class do the Pods of this lab have, and why?

Burstable: they set requests and limits, but the requests are lower than the limits.

▼ Why does `java-api` start with `-XX:MaxRAMPercentage=75` ?

So the heap is sized from the container's memory limit and leaves room for the JVM's non-heap memory. Without a sensible setting, the JVM could grow past the limit and be OOMKilled.

▼ You double the memory request of every service. What can go wrong on a small cluster?

Pods may stop fitting on the nodes: new Pods (or new replicas during a rolling update) stay Pending with "Insufficient memory", even though the real usage hasn't changed.

10 · Storage and databases

Time: 25 minutes

A container's files do not survive

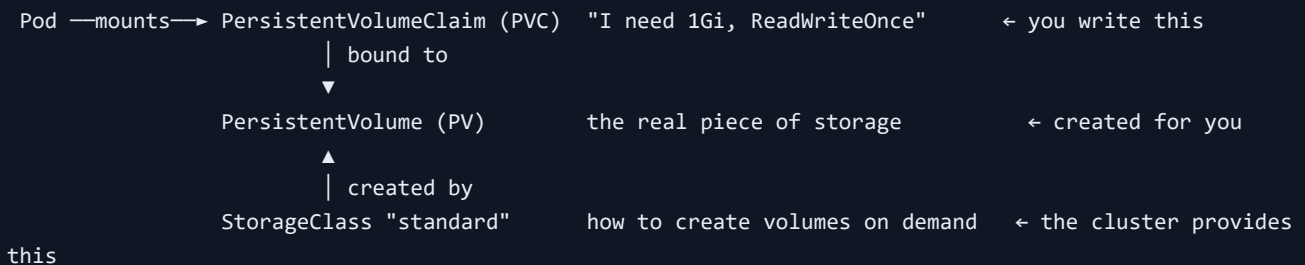
Everything a container writes to its own filesystem disappears when the container is replaced. In Kubernetes that happens all the time: a new image version, a node restart, a crash, a rescheduled Pod. For stateless services (`node-api` , `go-status` , `frontend` , ...) that is fine: their state lives in PostgreSQL. For **PostgreSQL itself** it would be a disaster.

In Compose, the database keeps its files in a named volume:

```
services:
  postgres:
    volumes:
      - pgdata:/var/lib/postgresql
volumes:
  pgdata: {}
```

Kubernetes has the same idea, split into three objects.

PV, PVC and StorageClass



Object	Who creates it	What it is
PersistentVolumeClaim	you (or a StatefulSet for you)	a request for storage: size and access mode
PersistentVolume	usually the StorageClass's provisioner	the actual storage: a cloud disk, an NFS share, a local directory
StorageClass	the cluster administrator	the recipe for creating PVs on demand

In this lab, kind ships a StorageClass called `standard` that uses the *local-path* provisioner: each volume is a directory on the kind node (a Docker container on your computer). Real clusters use cloud disks (for example EBS on AWS) through their CSI drivers. The manifests don't change: only the StorageClass behind them.

Access modes: `ReadWriteOnce` (one node mounts it read-write; normal for databases), `ReadOnlyMany` , `ReadWriteMany` (many nodes at once; needs shared storage such as NFS or EFS).

The lab's PostgreSQL: a StatefulSet

From [kubernetes/database/postgres.yaml](#):

```
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  ...
  volumeMounts:
    - name: data
      mountPath: /var/lib/postgresql
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

Why a **StatefulSet** and not a Deployment?

	Deployment	StatefulSet
Pod names	random (<code>node-api-6f678b5b5-kxsn1</code>)	stable and ordered (<code>postgres-0</code> , <code>postgres-1</code> , ...)
Storage	all replicas share whatever you mount	<code>volumeClaimTemplates</code> : each Pod gets its own PVC (<code>data-postgres-0</code>)
After a restart	a new Pod with a new name	the same name, re-attached to the same PVC
Use for	stateless services	databases, queues, anything with an identity and its own data

The Service in front of it is **headless** (`clusterIP: None`): the name `postgres` resolves straight to the Pod's IP. With several replicas, each one would also get its own DNS name (`postgres-0.postgres`), which database replication needs.

The `fsGroup: 70` in the Pod's security context makes the volume writable for the `postgres` user of the alpine image.

How long does the data live?

You delete ...	The data
the Pod <code>postgres-0</code>	survives: the StatefulSet recreates the Pod and re-attaches <code>data-postgres-0</code>
the StatefulSet	survives: the PVC is not deleted with it
the PVC <code>data-postgres-0</code>	gone (with the default reclaim policy <code>Delete</code> of local-path and most cloud StorageClasses)
the namespace <code>bookshop</code>	gone : the PVC is deleted with the namespace
the kind cluster	gone : the node containers, and the directories in them, are deleted

Databases need special care

Running PostgreSQL in Kubernetes is possible, and this lab does it so everything runs on one laptop. But be clear about what this lab's database **is not**:

- **No backups.** Nothing copies the data anywhere. One deleted PVC and it is lost.
- **No high availability.** One replica. If its node fails, the database is down until the Pod runs again.
- **No upgrade procedure.** A major version upgrade of PostgreSQL needs a data migration, not just a new image tag.
- **No tuning, monitoring or connection pooling.**

This single-replica PostgreSQL is fine for learning and not production-ready. In production, teams either use a **managed database** (Amazon RDS, Azure Database for PostgreSQL, Cloud SQL), which handles backups, failover and patching, or run PostgreSQL in Kubernetes with a database **operator** (such as CloudNativePG), which automates replication, backups and failover.

Check yourself

▼ Why doesn't `node-api` need a `PersistentVolumeClaim`?

It stores nothing on its own filesystem: its data is in PostgreSQL. Any replica can be replaced at any time without losing anything.

▼ What do `volumeClaimTemplates` give a `StatefulSet` that a `Deployment` doesn't have?

Each Pod gets its own PVC (`data-postgres-0` , `data-postgres-1` , ...), and a recreated Pod re-attaches to the same PVC.

▼ You run `kubectl delete pod postgres-0` . Is the data lost?

No. The `StatefulSet` creates `postgres-0` again and mounts the same PVC, `data-postgres-0` .

▼ What does the `StorageClass standard` do in kind, and what would replace it on AWS?

It creates volumes on demand as directories on the kind node (local-path provisioner). On AWS, a `StorageClass` using the EBS CSI driver creates EBS disks instead; the PVC stays the same.

▼ Name three things this lab's database lacks for production.

Backups, high availability (replication and failover), and an upgrade procedure. Also monitoring, tuning and connection pooling.

11 · Services, networking and Ingress

Time: 30 minutes

Pods come and go, and so do their IP addresses

Every Pod gets its own IP address. But Pods are replaced all the time (new versions, crashes, scaling), and each new Pod gets a **new** IP. Nobody can hard-code Pod IPs. A **Service** solves that.



A Service gives a group of Pods:

1. **a stable name:** `node-api`, resolvable by DNS inside the cluster,
2. **a stable virtual IP** (the ClusterIP) that never changes while the Service exists,
3. **load balancing** across its **endpoints**: the IPs of the Pods that match its selector **and are Ready**.

Labels and selectors: the only link

A Service finds its Pods through labels. From [kubernetes/node-api/](#):

```
# deployment.yaml (Pod template)
template:
  metadata:
    labels: { app: node-api, ... }

# service.yaml
spec:
  selector: { app: node-api }
  ports:
    - name: http
      port: 3000
      targetPort: http
```

If the selector and the labels differ by one character, the Service has **no endpoints**: no error anywhere, just "connection refused" or a timeout. `kubectl get endpoints node-api` (or `kubectl get endpointslices`) shows which Pod IPs a Service really sends traffic to. That check is in the troubleshooting labs.

`port` is the Service's port; `targetPort` is the container port, here referenced by its **name** `http`, so the number lives only in the Deployment.

DNS names

Inside the cluster, CoreDNS resolves Service names:

Name	Works from
<code>node-api</code>	Pods in the same namespace (<code>bookshop</code>)

Name	Works from
<code>node-api.bookshop</code>	any namespace
<code>node-api.bookshop.svc.cluster.local</code>	anywhere in the cluster (the fully qualified name)

That is why the Compose file and the Kubernetes manifests use the **same names**: `DB_HOST=postgres`, `http://python-api:8000/...` work unchanged in both. In Compose, Docker's DNS resolves service names; in Kubernetes, CoreDNS resolves Service names.

Service types

Type	Reachable from	In this lab
ClusterIP (default)	inside the cluster only	every application Service
NodePort	every node's IP on a port 30000–32767	the Traefik controller, on 30080
LoadBalancer	an external load balancer created by the cloud provider	not used (kind has no cloud)
headless (<code>clusterIP: None</code>)	DNS returns the Pod IPs directly	<code>postgres</code>

Ingress: one entry point for HTTP

Exposing every service with its own NodePort would be messy. An **Ingress** is a set of HTTP routing rules: host name and path → Service. From [kubernetes/ingress/ingress.yaml](#):

```

http://bookshop.localhost:8080/api/users → node-api:3000
http://bookshop.localhost:8080/api/books → java-api:8080
http://bookshop.localhost:8080/api/stats → python-api:8000
http://bookshop.localhost:8080/api/status → go-status:8080
http://bookshop.localhost:8080/ → frontend:8080 (path-based rules, one host)
http://admin.bookshop.localhost:8080/ → laravel-admin:8080 (host-based rule)

```

The most specific path wins: `/api/users` goes to `node-api`, everything else under `/` to the frontend.

An Ingress object alone does nothing. An **ingress controller** reads the Ingress objects and does the routing. This lab uses **Traefik**, a maintained controller, installed with Helm ([kubernetes/cluster/traefik-values.yaml](#)); `ingressClassName: traefik` selects it. Other controllers (cloud load balancer controllers, HAProxy, Kong, ...) read the same Ingress objects.

How a request from your browser arrives

```

browser: http://bookshop.localhost:8080/api/books
|
|  (*.localhost always resolves to 127.0.0.1)
|
▼
your computer, port 8080
|
|  kind extraPortMappings: hostPort 8080 → node port 30080    (kubernetes/cluster/kind-config.yaml)
|
▼
kind node (a Docker container), port 30080
|
|  Service traefik, type NodePort 30080
|
▼
Traefik Pod: matches host bookshop.localhost + path /api/books
|
▼
Service java-api:8080 → a Ready java-api Pod

```

In Compose, the frontend's nginx did this routing (it proxies `/api/...` to the services). In Kubernetes the Ingress does it, so each API is reached directly.

Gateway API, the successor

The Ingress API is stable but limited (HTTP routing only; anything more needs controller-specific annotations). The **Gateway API** is its successor in Kubernetes: `GatewayClass`, `Gateway` and `HTTPRoute` objects split the work between the platform team (who runs the gateway) and application teams (who own their routes), and it supports more protocols and features in a portable way. Traefik supports it too. The concepts of this lesson (host, path, backend Service) carry over directly.

Check yourself

▼ Why can't frontend call node-api by its Pod IP?

Pod IPs change whenever a Pod is replaced. The Service gives a stable name and virtual IP that always point at the current Ready Pods.

▼ A Service exists, the Pods run, but `kubectl get endpoints` shows `<none>`. What is wrong?

The Service's selector doesn't match the Pods' labels, or the Pods are not Ready (failing readiness probe).

▼ Which DNS name works from another namespace: `node-api` or `node-api.bookshop`?

`node-api.bookshop` (or the full `node-api.bookshop.svc.cluster.local`). The short name only works in the same namespace.

▼ What does the Traefik controller do with the Ingress object?

It watches Ingress objects and configures itself to route each host and path to the right Service. Without a controller, the Ingress object has no effect.

▼ Trace `http://admin.bookshop.localhost:8080/` from the browser to a container.

Port 8080 on your computer → kind port mapping → node port 30080 → Traefik Service (NodePort) → Traefik matches the host `admin.bookshop.localhost` → Service `laravel-admin:8080` → the `laravel-web` container (nginx) → PHP-FPM in the same Pod.

12 · Scaling, rolling updates and rollbacks

Time: 25 minutes

Replicas

A Deployment keeps a number of identical Pods running: `replicas: 2` in [kubernetes/node-api/deployment.yaml](#). More replicas give you:

- **availability**: one Pod crashes or its node fails, the others keep serving;
- **capacity**: the Service spreads requests over all Ready Pods;
- **zero-downtime updates**: there is always an old Pod serving while a new one starts.

Change the number on the fly:

```
kubectl -n bookshop scale deployment node-api --replicas=3
kubectl -n bookshop get pods -l app=node-api
```

The Service picks up the new Pod as soon as it is **Ready**. Note: `kubectl scale` changes the live object only. The next `kubectl apply` of the YAML sets the replicas back to the file's value, so permanent changes belong in the file.

How a Deployment changes versions: ReplicaSets

A Deployment does not manage Pods directly. It manages **ReplicaSets**, one per version of the Pod template:

```
Deployment node-api
├─ ReplicaSet node-api-6f678b5b5 (image node-api:1.0.0) revision 1
└─ ReplicaSet node-api-7c9d8f4b6 (image node-api:1.1.0) revision 2 ← current
```

Every change to the Pod template (image, env, probes, resources) creates a new ReplicaSet: a new **revision**. Old ReplicaSets are kept (scaled to 0), which is what makes a rollback instant.

Rolling update

The default strategy is `RollingUpdate`: replace Pods a few at a time.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%      ← how many extra Pods above "replicas" may exist during the update (rounded up)
    maxUnavailable: 25% ← how many Pods below "replicas" may be unavailable (rounded down)
```

With `replicas: 2`: `maxSurge 25 % of 2` → rounded **up** to 1, `maxUnavailable 25 % of 2` → rounded **down** to 0. So Kubernetes starts **one** new Pod, waits until it is **Ready**, removes one old Pod, and repeats. At no moment are fewer than 2 Pods serving.

```

v1 v1      start
v1 v1 v2    new Pod starting (surge)
v1 v1 v2✓   new Pod Ready
v1 v2✓      one old Pod removed
v1 v2✓ v2   next new Pod
v2✓ v2✓     done

```

Readiness gates the rollout. A new Pod that never becomes Ready stops the rollout: the old Pods keep serving and `kubectl rollout status` waits (and eventually reports `exceeded its progress deadline`). A bad version with a good readiness probe never takes all traffic.

The commands

Command	What it does
<code>kubectl -n bookshop set image deployment/node-api node-api=node-api:1.1.0</code>	changes the image (container name = image reference): starts a rolling update
<code>kubectl -n bookshop rollout status deployment/node-api</code>	waits until the rollout has finished (or failed)
<code>kubectl -n bookshop rollout history deployment/node-api</code>	lists the revisions
<code>kubectl -n bookshop rollout undo deployment/node-api</code>	goes back to the previous revision
<code>kubectl -n bookshop rollout undo deployment/node-api --to-revision=1</code>	goes back to a specific revision
<code>kubectl -n bookshop annotate deployment/node-api kubernetes.io/change-cause="1.1.0: ..."</code>	records why : shown in the CHANGE-CAUSE column of <code>rollout history</code>

A rollback is itself a rollout: the old ReplicaSet scales up, the new one down, with the same surge rules.

Versioned tags make rollbacks meaningful

Revisions record the **image reference**. If you deploy `node-api:latest`, then build a new `latest` and roll back, the "old" revision still says `node-api:latest`, which now means the new, broken image (or whatever the node has cached). With `node-api:1.0.0` and `node-api:1.1.0`, a rollback goes back to exactly the image that worked. That is why this lab never uses `latest` ([docs/04](#)).

The next step: autoscaling

`kubectl scale` is manual. A **HorizontalPodAutoscaler** (HPA) changes `replicas` automatically, for example to keep average CPU usage around 70 % of the requests, between a minimum and a maximum number of Pods. It needs the metrics-server (for CPU and memory) and sensible CPU requests, because it calculates utilisation relative to them. This lab scales by hand so you can see each step.

Check yourself

▼ You scaled `node-api` to 3 with `kubectl scale`, then ran `kubectl apply -f kubernetes/node-api/`.
How many replicas now?

2, the value in the file. `kubectl scale` only changed the live object.

▼ With `replicas: 2` and the default strategy, how many Pods run at most during a rolling update, and how few are Ready at least?

At most 3 (`maxSurge` rounds up to 1); at least 2 Ready (`maxUnavailable` rounds down to 0).

▼ A new version's readiness probe always fails. What happens to the traffic?

Nothing changes for users: the new Pod never becomes Ready, so it gets no traffic, and the old Pods are not removed. The rollout stalls until you fix it or roll back.

▼ Why does `kubectl rollout undo` work instantly?

The previous ReplicaSet still exists (scaled to 0). Undo scales it up again; no new template has to be built.

▼ Why is rolling back between two `latest` images unreliable?

Both revisions record the same reference, `latest`, so the "previous" revision doesn't identify the image that worked.

13 · Logging and debugging

Time: 30 minutes

Logs go to stdout and stderr

Every application in this lab writes its logs to **standard output** (stdout) or **standard error** (stderr), never to files ([docs/CONTRACT.md](#), rule 6). The container runtime captures those two streams; `docker logs` reads them in Docker and Compose, `kubectl logs` reads them in Kubernetes. Log files inside a container would be invisible to both, and lost when the container is replaced.

kubectl logs

Command	Shows
<code>kubectl -n bookshop logs deploy/node-api</code>	the logs of one Pod of the Deployment
<code>kubectl -n bookshop logs <pod></code>	the logs of one specific Pod
<code>kubectl -n bookshop logs -f <pod></code>	follow: new lines as they arrive (Ctrl+C to stop)
<code>kubectl -n bookshop logs <pod> --previous</code>	the logs of the previous , crashed container: the most important flag for <code>CrashLoopBackOff</code>
<code>kubectl -n bookshop logs <pod> -c laravel-fpm</code>	one container of a multi-container Pod (<code>laravel-admin</code> has <code>laravel-web</code> and <code>laravel-fpm</code>)
<code>kubectl -n bookshop logs -l app=python-api --prefix</code>	all Pods with that label, each line prefixed with its Pod
<code>kubectl -n bookshop logs job/report-now</code>	the logs of a Job's Pod
<code>kubectl -n bookshop logs <pod> --since=10m --tail=50</code>	only recent lines

Logs disappear with the Pod. Production clusters ship them to a central system (Loki, Elasticsearch, CloudWatch, ...) so they survive and can be searched.

The debugging commands and the question each one answers

Command	Answers
<code>kubectl get pods</code>	does it run? What is the STATUS (Pending, ContainerCreating, ImagePullBackOff, CrashLoopBackOff, Running) and READY (<code>1/1</code>)? How many RESTARTS?
<code>kubectl get pods -o wide</code>	on which node, with which Pod IP?
<code>kubectl describe pod <pod></code>	why : container state and last state (exit code, <code>OOMKilled</code>), probe results, and the Events at the bottom (scheduling, image pulls, probe failures)

Command	Answers
<code>kubectl logs <pod></code>	what did the application itself say?
<code>kubectl exec -it <pod> -- sh</code>	look from inside: does the app answer on localhost? Is the env variable set? (not possible with distroless images like <code>go-status</code> , which have no shell)
<code>kubectl get svc</code>	which Services exist, their type, ClusterIP and ports
<code>kubectl describe svc <svc></code>	the selector, ports, targetPort and the current endpoints
<code>kubectl get endpoints <svc> / kubectl get endpointslices</code>	which Pod IPs the Service really sends traffic to (<code><none></code> = selector mismatch or no Ready Pod)
<code>kubectl get events --sort-by=.lastTimestamp</code>	everything that happened in the namespace, newest last
<code>kubectl get deployments</code>	READY / UP-TO-DATE / AVAILABLE replicas
<code>kubectl rollout status deployment/<d></code>	is the rollout progressing, finished or stuck?
<code>kubectl rollout history deployment/<d></code>	which revisions exist, for <code>rollout undo</code>

A decision flow

Start with `kubectl get pods` and follow the status:

```

kubect1 get pods
├── Pending ────────────> kubect1 describe pod → Events
                        "Insufficient cpu/memory" (requests), "unbound PersistentVolumeClaim"
                        (storage),
                        "untolerated taint", node selector ...
├── ErrImagePull / ImagePullBackOff
                        ──> kubect1 describe pod → Events: wrong image name or tag? image not in the
registry
                        (or not loaded into kind)? private registry without credentials?
├── CrashLoopBackOff ──> kubect1 logs <pod> --previous (the crash reason)
                        kubect1 describe pod → Last State: exit code, OOMKilled?
                        typical: missing env variable or Secret, wrong config, out of memory
├── CreateContainerConfigError
                        ──> kubect1 describe pod → a referenced ConfigMap/Secret or key does not exist
├── Running but READY 0/1 ▶ kubect1 describe pod → "Readiness probe failed: ..."
                        wrong probe path/port? does the dependency (database) answer?
└── Running and Ready, but requests fail
    ├── inside the cluster ▶ kubect1 get endpoints <svc>: <none>? → selector/labels mismatch
    │                       kubect1 describe svc <svc>: port / targetPort match the container port?
    │                       test from another Pod: kubect1 run tmp --rm -it --image=busybox:1.37 --
wget -q0- http://<svc>:<port>/health
    │
    └── only from outside ▶ the Ingress: kubect1 describe ingress bookshop (host, path, backend
                        Service and port), the ingress controller's logs (kubect1 -n traefik
logs deploy/traefik)

```

The [troubleshooting labs](#) walk through each branch with a real failure.

Check yourself

▼ A Pod is in CrashLoopBackOff and `kubect1 logs` shows nothing useful. What next?

`kubect1 logs <pod> --previous` : the current container may just have started; the previous one holds the crash message. Then `kubect1 describe pod` for the exit code and reason (for example OOMKilled).

▼ How do you read the logs of PHP-FPM in the `laravel-admin` Pod?

`kubect1 -n bookshop logs <laravel-admin-pod> -c laravel-fpm` : name the container, because the Pod has two.

▼ Everything is Running and Ready, but `curl http://node-api:3000/health` from another Pod fails. Which command first?

`kubect1 get endpoints node-api` . If it shows `<none>` , the Service's selector doesn't match the Pods' labels.

▼ Why can't you `kubect1 exec ... -- sh` into the `go-status` Pod?

Its image is distroless: it contains only the Go binary, no shell. Debug it with its logs, from another Pod, or with `kubectl debug` and an ephemeral container.

▼ **Why should applications log to stdout instead of files?**

The container runtime captures stdout/stderr, so `kubectl logs` and log collectors can read them. Files inside the container are invisible to them and vanish with the container.

14 · Jobs and CronJobs

Time: 20 minutes

Three kinds of work

Workload	Runs	When it ends	In this lab
Deployment	forever; restarted whenever it stops	never (until you delete it)	node-api , python-api , java-api , go-status , frontend , laravel-admin
Job	until it succeeds (exit code 0), retried on failure	when it has completed	laravel-migrate (database migrations)
CronJob	creates a Job on a schedule	each Job ends; the CronJob stays	report-worker

A server that exits is a problem: a Deployment restarts it. A batch task that exits with 0 is **done**: it must not be restarted. That is why `report-worker` (it collects statistics, stores one row, and exits) is a CronJob, not a Deployment. In Compose, the same distinction is a service you start with `docker compose run --rm report-worker` (and `profiles: [jobs]`, so `docker compose up` doesn't start it).

The report-worker CronJob

From [kubernetes/report-worker/cronjob.yaml](#):

```
spec:
  schedule: "*/10 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      ttlSecondsAfterFinished: 3600
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report-worker
              image: report-worker:1.0.0
```

Field	Meaning
<code>schedule: "*/10 * * * *"</code>	cron syntax: minute, hour, day of month, month, day of week. <code>*/10</code> in the minute field = every 10 minutes
<code>concurrencyPolicy: Forbid</code>	if the previous run is still going, skip this one (<code>Allow</code> would run both; <code>Replace</code> would stop the old one)
<code>successfulJobsHistoryLimit / failedJobsHistoryLimit</code>	how many finished Jobs (and their Pods, with their logs) to keep

Field	Meaning
jobTemplate	the Job that is created at each scheduled time
backoffLimit: 2	a failing run is retried, then the Job is marked Failed
ttlSecondsAfterFinished: 3600	delete the finished Job (and its Pods) an hour later
restartPolicy: OnFailure	restart the container in the same Pod if it fails. Jobs allow only OnFailure or Never, never Always

Don't wait 10 minutes to test it. Create a Job from the CronJob's template, right now:

```
kubectl -n bookshop create job report-now --from=cronjob/report-worker
kubectl -n bookshop wait --for=condition=complete job/report-now --timeout=90s
kubectl -n bookshop logs job/report-now
```

The laravel-migrate Job

Laravel's database migrations must run **once per release**, before the new version serves traffic. Not in every replica at startup: two replicas starting at the same time would run the same migration twice. From [kubernetes/laravel-admin/migrate-job.yaml](#):

```
spec:
  backoffLimit: 4 # the database may still be starting: retry a few times
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: laravel-fpm:1.0.0
          args: ["php", "artisan", "migrate", "--force", "--seed"]
```

Field	Why
image: laravel-fpm:1.0.0	the same image as the application: the migrations are part of the code of that release
args: (not command:)	replaces only the CMD, so the image's entrypoint still runs and checks that APP_KEY and DB_PASSWORD are set
restartPolicy: Never + backoffLimit: 4	each retry is a new Pod (each failed Pod stays for inspection), up to 4 retries
--force	Laravel refuses to migrate in production without it

A Job's spec cannot be changed after it is created. For the next release, delete the old Job (or let `ttlSecondsAfterFinished` remove it) and apply the new one. Deployment tools such as Helm hooks or Argo CD sync waves automate "migrate first, then roll out".

Check yourself

▼ Why is `report-worker` a CronJob and not a Deployment?

It finishes its work and exits with 0. A Deployment would restart it immediately, forever. A CronJob runs it as a Job on a schedule, and a completed Job is not restarted.

▼ What does `concurrencyPolicy: Forbid` prevent?

Two report runs at the same time: if the previous run hasn't finished when the next scheduled time arrives, the new run is skipped.

▼ How do you run the CronJob's work once, now?

```
kubect1 -n bookshop create job report-now --from=cronjob/report-worker .
```

▼ Why don't the Laravel Pods run their migrations at startup?

Every replica would run them, possibly at the same time. A Job runs them once per release, before the new version serves traffic.

▼ Which `restartPolicy` values can a Job's Pod use?

`OnFailure` or `Never`. `Always` is only for long-running workloads such as Deployments.

15 · Multi-container Pods

Time: 20 minutes

Why Laravel is two programs

A Node.js, Python, Go or Java API is **one process** that both speaks HTTP and runs the application. PHP traditionally works differently:

```
browser —HTTP→ nginx —FastCGI→ PHP-FPM —runs→ Laravel (index.php)
                web server      PHP process manager
                - static files   - a pool of PHP worker processes
                - HTTP, buffering - runs one PHP request per worker
```

- **nginx** handles HTTP: it serves CSS, JS and images directly and forwards only `*.php` requests.
- **PHP-FPM** (FastCGI Process Manager) keeps a pool of PHP processes and runs the PHP code. It does not speak HTTP; it speaks **FastCGI**, on port 9000.

Two programs, two images: `laravel-web:1.0.0` (nginx with the `public/` assets) and `laravel-fpm:1.0.0` (PHP 8.5 with the application). One process per container keeps each image small, lets each be updated on its own, and gives each its own logs and health check.

In Compose: two services

From `compose/docker-compose.yml`:

```
laravel-fpm:
  image: laravel-fpm:1.0.0
  ...
laravel-web:
  image: laravel-web:1.0.0
  environment:
    FPM_HOST: laravel-fpm      ← nginx reaches FPM over the Compose network, by service name
  ports: ["127.0.0.1:8082:8080"]
```

In Kubernetes: one Pod, two containers

From `kubernetes/laravel-admin/deployment.yaml`:

```
spec:
  containers:
    - name: laravel-web
      image: laravel-web:1.0.0
      env:
        - name: FPM_HOST
          value: "127.0.0.1" # the other container of this Pod
    - name: laravel-fpm
      image: laravel-fpm:1.0.0
      ports:
        - name: fastcgi
          containerPort: 9000
```

All containers of a Pod share **one network namespace**: one IP address, one `localhost`. nginx reaches PHP-FPM at `127.0.0.1:9000` without any Service. They are also scheduled together (same node), scaled together (a replica is the pair) and started and stopped together.

```
Pod laravel-admin-... (one Pod IP, one localhost)
  laravel-web (nginx :8080) —127.0.0.1:9000→ laravel-fpm (php-fpm :9000)
    ▲
Service laravel-admin:8080 → only port 8080 is exposed; FPM is reachable only inside the Pod
```

Sidecar in one Pod, or two Deployments?

Both designs work; they make different trade-offs.

	One Pod, two containers (this lab)	Two Deployments + a Service for FPM
Communication	<code>127.0.0.1</code> , no Service, no network hop	over the cluster network, through a Service
Scaling	together: every replica is one nginx + one FPM	independently (e.g. 2 nginx, 6 FPM)
Updates	one rollout replaces both	separately
Fits when	the two are tightly coupled and always used as a pair	they scale very differently, or one serves several others

Rule of thumb: put containers in one Pod only when they **must** run together on the same machine and share its network (or files). Otherwise, give each its own Deployment. Classic sidecars: a web server in front of an app server (this lab), a log shipper, a proxy of a service mesh.

Init containers and native sidecars

A Pod can also have **init containers**: they run to completion, one after another, **before** the main containers start (for example: wait until the database answers, or prepare files in a shared volume). Since Kubernetes 1.29, an init container with `restartPolicy: Always` is a **native sidecar**: it starts before the main containers, keeps running alongside them, and is stopped after them, which fixes ordering problems of classic sidecars (for example in Jobs, where a classic sidecar would keep the Pod from ever completing).

Probing a container that doesn't speak HTTP

nginx has an HTTP probe on `/health` and `/ready`. PHP-FPM speaks FastCGI, not HTTP, so an `httpGet` probe can't talk to it. Its liveness probe only checks that it accepts TCP connections:

```
livenessProbe:
  tcpSocket: { port: fastcgi }      # FPM speaks FastCGI, not HTTP: check that it accepts
connections
  periodSeconds: 10
```

The readiness of the whole Pod is checked through nginx: `laravel-web`'s `/ready` passes through PHP-FPM to Laravel, which checks the database. One request tests nginx, FPM, Laravel and PostgreSQL. A Pod is Ready only when **all** its containers are Ready.

Check yourself

▼ Why does `laravel-web` use `FPM_HOST=127.0.0.1` in Kubernetes but `laravel-fpm` in Compose?

In Kubernetes both containers are in one Pod and share its network namespace, so FPM is on localhost. In Compose they are separate containers on a network, reached by service name.

▼ What do you lose by putting nginx and PHP-FPM in one Pod?

Independent scaling and independent updates: each replica is always one nginx plus one FPM, and both are replaced in the same rollout.

▼ Why is FPM's liveness probe `tcpSocket` instead of `httpGet` ?

PHP-FPM speaks FastCGI, not HTTP. A TCP check verifies it accepts connections; the HTTP checks go through nginx.

▼ When is a Pod with two containers Ready?

When every one of its containers is Ready. Only then does the Service send it traffic.

▼ What runs before the main containers of a Pod start?

The init containers, in order, each to completion. Native sidecars (init containers with `restartPolicy: Always`) start before the main containers and keep running.

Service contract

Every application in this lab follows the same small contract. This is what makes them interchangeable for Docker, Compose and Kubernetes: **Kubernetes does not care which language a container was written in, only how it behaves.**

The platform: "Bookshop"

Service (name = Compose service = Kubernetes Service)	Stack	Port	What it does	Talks to
frontend	React 19 + Vite 8, served by nginx (unprivileged)	8080	the web UI: books, users, statistics, service status	the APIs, through /api/...
node-api	Node.js 24 + Express 5	3000	users: GET/POST /api/users	PostgreSQL
python-api	Python 3.14 + FastAPI	8000	statistics: GET /api/stats (counts + latest report)	PostgreSQL (read only)
go-status	Go 1.27, standard library only	8080	service status board: GET /api/status checks every service's /health	all services over HTTP
java-api	Java 25 + Spring Boot 4.1	8080	books: GET /api/books , GET /api/books/{id}	PostgreSQL
laravel-admin	PHP 8.5 + Laravel 13: laravel-fpm (PHP-FPM) + laravel-web (nginx)	8080 (nginx) / 9000 (FPM)	book reviews: list and add (server-rendered HTML)	PostgreSQL
report-worker	JavaScript (plain, no framework) on Node.js 24	–	a batch job: collects stats and status, stores a report row, exits	python-api , go-status , PostgreSQL
postgres	PostgreSQL 18	5432	the database bookshop	–

Rules for every HTTP service

1. **Port from the environment:** listen on `PORT` (default = the port in the table), on `0.0.0.0`.
2. **GET /health** returns HTTP 200 and JSON `{"status":"ok","service":"<name>","version":"<version>"}` as soon as the process can serve requests. It does **not** check the database (a database outage must not make Kubernetes restart every API). Services with a database also expose **GET /ready** : 200 `{"status":"ready",...}` when the database answers, 503 `{"status":"not ready","reason":"..."}` when it does not. (`/health` → liveness, `/ready` → readiness.)
3. **GET /** returns JSON with `service`, `version`, `language`, `runtime` and a one-line `description`.
4. **Version:** baked in at build time (`ARG APP_VERSION=1.0.0` → `env APP_VERSION`). The image tag equals the version.
5. **Configuration only from environment variables.** Database: `DB_HOST` (default `postgres`), `DB_PORT` (5432), `DB_NAME` (`bookshop`), `DB_USER` (`bookshop`), `DB_PASSWORD` (**required**, no default: a missing

password must produce a clear error message and exit code 1 at startup).

6. **Logs:** one line per request and per important event, to **stdout** (never to files).

7. **Graceful shutdown** on SIGTERM.

8. **Non-root user** in the image, no secrets in the image, versioned base images, small final images (multi-stage where a build step exists), a `.dockerignore`.

9. **Database tables:** each service creates the tables it owns if they do not exist and seeds a few rows:

`node-api` → `users`, `java-api` → `books`, `laravel-admin` → `reviews` (Laravel migrations, run as a separate step), `report-worker` → `reports`. `python-api` only reads (a missing table counts as 0).

Endpoints

Service	Endpoints
<code>node-api</code>	<code>/</code> , <code>/health</code> , <code>/ready</code> , <code>GET /api/users</code> , <code>POST /api/users</code> (<code>{"name","email"}</code>)
<code>python-api</code>	<code>/</code> , <code>/health</code> , <code>/ready</code> , <code>GET /api/stats</code> → <code>{"users":n,"books":n,"reviews":n,"latest_report":{...} null}</code>
<code>go-status</code>	<code>/</code> , <code>/health</code> , <code>GET /api/status</code> → <code>{"services":</code> <code>[{"name","url","status":"up down","http_status","latency_ms"}], "up":n, "total":n}</code> ; targets from <code>TARGETS = name=url,name=url</code>
<code>java-api</code>	<code>/</code> , <code>/health</code> , <code>/ready</code> , <code>GET /api/books</code> , <code>GET /api/books/{id}</code>
<code>laravel-admin</code>	<code>/</code> (reviews page + form), <code>POST /reviews</code> , <code>/health</code> , <code>/ready</code>
<code>frontend</code>	<code>/</code> (the app), <code>/health</code> ; in Compose its nginx also proxies <code>/api/users</code> , <code>/api/books</code> , <code>/api/stats</code> , <code>/api/status</code> to the services (in Kubernetes the Ingress does that routing)

Images

Local names `<service>:<version>`, for example `node-api:1.0.0` ; published to GHCR as `ghcr.io/sufyanahmadkamboh/bookshop-<service>:<version>`. Never `latest`.

Troubleshooting labs

Level 19 of the [roadmap](#). You need the complete platform from the [Kubernetes lessons](#), running in the namespace `bookshop` (the current namespace of your `kubectl` context).

Each lab **breaks one thing on purpose**, shows the symptom exactly as you would meet it at work, and walks through the investigation. Every lab ends by restoring the platform, so you can do them in any order, and repeat them. All outputs are real: they were recorded while the labs ran against the lab cluster.

Break it → Problem → Symptoms → Investigation → Commands → Root cause → Fix → Verification → Lesson learned

#	Lab	What breaks	Typical symptom	Key commands
01	Wrong image name	a typo in the image name (go-status)	<code>ImagePullBackOff</code>	<code>describe pod</code> , <code>get events</code>
02	Image pull failure	an image built locally but never given to the cluster (python-api)	<code>ImagePullBackOff</code> with a correct name	<code>docker images</code> , <code>crictl images</code> on the node
03	Wrong container port	the app listens on another port than the manifest says (node-api)	Running, <code>0/1</code> , restarts	<code>get events</code> , <code>exec ... wget 127.0.0.1</code>
04	Service selector mismatch	a typo in the Service selector (node-api)	Ingress 503, healthy Pods	<code>get endpointslices</code> , <code>--show-labels</code>
05	Application crashes	a wrong start command (node-api)	<code>CrashLoopBackOff</code>	<code>logs --previous</code> , <code>describe</code> → Last State
06	Missing environment variable	<code>DB_PASSWORD</code> removed (python-api)	<code>CrashLoopBackOff</code> with a clear message	<code>logs --previous</code> , the env list
07	Database connection failure	<code>DB_HOST</code> typo in the ConfigMap (node-api)	Running, <code>0/1</code> , no restarts	<code>/ready</code> from inside, <code>printenv</code>
08	Readiness probe failure	readiness path typo (python-api)	stuck rollout, <code>0/1</code>	<code>rollout status</code> , <code>probe events</code>
09	Volume problem	a claim with a non-existent StorageClass	<code>Pending</code>	Pod events, <code>describe pvc</code> , <code>get storageclass</code>
10	Works in the Pod, not through the Service	wrong <code>targetPort</code> (python-api)	Ingress 502	hop-by-hop tests, <code>describe service</code>
11	Frontend cannot reach a backend	the java-api Service port changed	UI panel error, 502 from the wrong component	<code>curl -i headers</code> , <code>describe ingress</code>
12	Incorrect Secret	a "rotated" password the database doesn't know	Running, <code>0/1</code>	<code>/ready</code> , checksum comparison

The method

- | | |
|--|--|
| 1. What exactly is the symptom?
the Ingress | <code>kubectl get pods (STATUS, READY, RESTARTS) · curl -i through</code> |
| 2. What does Kubernetes know? | <code>kubectl describe pod / service / ingress · kubectl get events</code> |
| 3. What does the application say? | <code>kubectl logs [--previous] · its /health and /ready answers</code> |
| 4. Which hop fails?
Ingress | inside the Pod (127.0.0.1) → through the Service → through the |
| 5. Change one thing, then verify | the command that showed the symptom must now show it fixed |

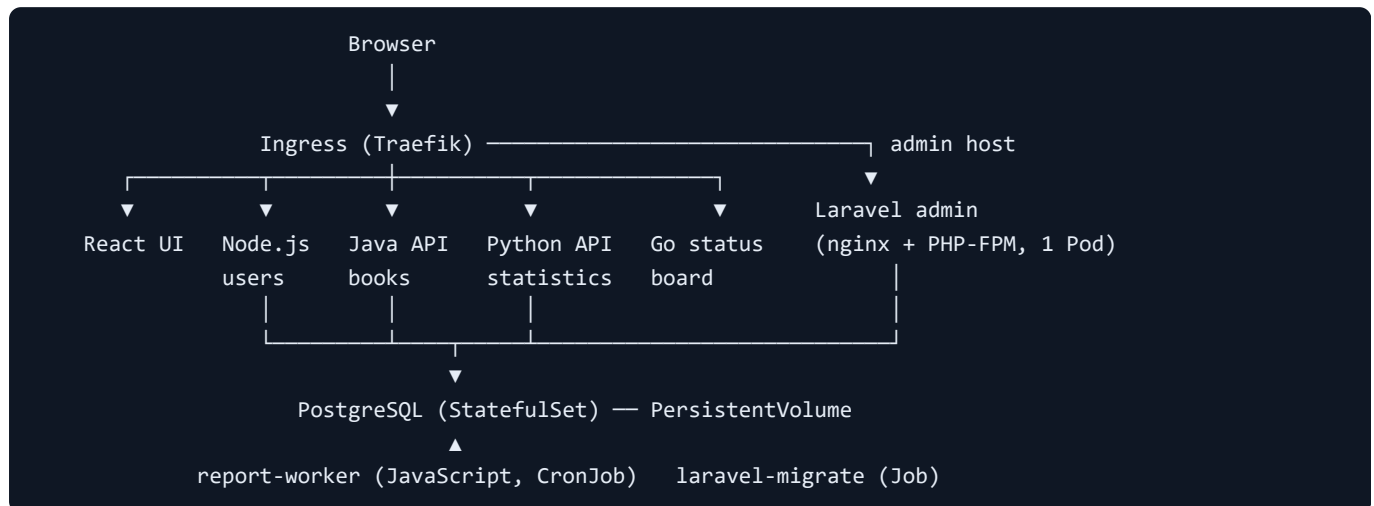
Status	First command	Usual causes
Pending	<code>kubectl describe pod (Events)</code>	no node with enough CPU/memory, unbound volume claim, taints
ErrImagePull / ImagePullBackOff	<code>kubectl describe pod / events</code>	wrong name or tag, image not in a reachable registry, missing pull credentials
CreateContainerConfigError	<code>kubectl describe pod</code>	a referenced ConfigMap/Secret or key does not exist
CrashLoopBackOff	<code>kubectl logs <pod> --previous</code>	the program exits: bad command, missing configuration, crash at startup
Running but 0/1	events (readiness probe), <code>/ready</code>	dependency down, wrong probe path/port, app listens elsewhere
Running, 1/1, but unreachable	<code>kubectl get endpointslices, describe service</code>	selector mismatch, wrong <code>targetPort</code> , Ingress rule
OOMKilled (exit 137)	<code>kubectl describe pod → Last State</code>	memory limit too low (kubernetes/13)

The command reference is in [docs/13 · Logging and debugging](#).

Capstone · the complete multi-stack platform

Level 20 of the [roadmap](#). Time: 1–2 hours.

Everything the course built, together, verified by a script. Then a final on-call scenario.



What the finished platform must demonstrate, and how it is checked:

Capability	Where it lives	Checked by <code>verify.sh</code>
multiple languages, Dockerfiles, versioned images	applications/, <code><service>:1.0.0</code>	no image without a version tag
Docker Compose	compose/	(the Compose lesson)
Deployments, Services, Ingress	kubernetes/*/, ingress/	all Available, every route answers through the Ingress
ConfigMaps, Secrets	config/, secrets created by kubectl	the password is not in the ConfigMap
health probes, startup probe	every Deployment	liveness/readiness everywhere, startupProbe on java-api
requests and limits	every container	CPU and memory requests and limits everywhere
storage	the PostgreSQL StatefulSet	claim Bound; data survives deleting the database Pod
Jobs and CronJobs, multi-container Pods	laravel-migrate, report-worker, laravel-admin	Job complete, CronJob exists, 2 containers
rolling updates done	every Deployment	no Deployment stuck mid-rollout
security basics	every Pod	non-root everywhere

Part A · Deploy it all, from nothing

`deploy.sh` does what lessons 00–09 did, in the right order, in one command: build the images, create the cluster and the ingress controller if they do not exist, load the images, create the namespace, configuration

and generated Secrets, start the database, run the migration Job, deploy every application, create the Ingress, and run the first report. Read it first: every step is one you know. It is idempotent: running it again changes nothing that is already correct.

```
bash capstone/deploy.sh
```

```
...
deployment.apps/node-api condition met
deployment.apps/python-api condition met

==> 7/8 entry point
ingress.networking.k8s.io/bookshop unchanged

==> 8/8 first report
job.batch/capstone-report created
job.batch/capstone-report condition met

Bookshop is up: http://bookshop.localhost:8080    admin: http://admin.bookshop.localhost:8080
Verify it:    capstone/verify.sh
```

Part B · Verify everything

```
bash capstone/verify.sh
```

```
Workloads
PASS 6 Deployments, all Available
PASS no Deployment is stuck in the middle of a rollout
PASS PostgreSQL StatefulSet ready (1/1)
PASS database volume claim Bound
PASS migration Job completed
PASS report CronJob exists
PASS laravel-admin Pod runs 2 containers
Good practice in every Deployment
PASS every container has CPU and memory requests and limits
PASS every HTTP container has a liveness probe
PASS every Service-facing container has a readiness probe
PASS java-api has a startupProbe
PASS no image uses the tag latest (or no tag)
PASS every Pod runs as non-root
PASS the database password comes from a Secret, not a ConfigMap
Through the Ingress
PASS UI: /
PASS Node.js: /api/users
PASS Java: /api/books
PASS Python: /api/stats (with a report)
PASS Go: /api/status, every service up
PASS Laravel: admin host
Resilience
PASS data survives the loss of the database Pod
PASS every Deployment Available again afterwards

22 passed, 0 failed
```

Part C · The final challenge: Monday morning

Over the weekend, three changes were made to the running platform. Users report that statistics are missing and the status board looks odd. **Run `break.sh` without reading it**, then find and fix every problem using only what the course taught (`get`, `describe`, `logs`, endpoints, events, rollout history), until `verify.sh` passes again.

```
bash capstone/break.sh
```

```
bash capstone/verify.sh
```

```
Workloads
PASS 6 Deployments, all Available
FAIL no Deployment is stuck in the middle of a rollout
['go-status', 'java-api']
PASS PostgreSQL StatefulSet ready (1/1)
PASS database volume claim Bound
PASS migration Job completed
PASS report CronJob exists
PASS laravel-admin Pod runs 2 containers
Good practice in every Deployment
PASS every container has CPU and memory requests and limits
PASS every HTTP container has a liveness probe
PASS every Service-facing container has a readiness probe
PASS java-api has a startupProbe
PASS no image uses the tag latest (or no tag)
PASS every Pod runs as non-root
PASS the database password comes from a Secret, not a ConfigMap
Through the Ingress
PASS UI: /
PASS Node.js: /api/users
PASS Java: /api/books
FAIL Python: /api/stats (with a report)
      raise JSONDecodeError("Expecting value", s, err.value) from None
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
FAIL Go: /api/status, every service up
      5 6
PASS Laravel: admin host
Resilience
PASS data survives the loss of the database Pod
PASS every Deployment Available again afterwards

19 passed, 3 failed
```

Rules: fix the cause, not the symptom (no deleting and recreating everything), and write down for each problem: symptom → command that showed the cause → root cause → fix.

▼ Solution

1. **Statistics missing.** `/api/stats` returns 503 no available server. The Pods of python-api are Running and Ready, but `kubectl get endpointslices -l kubernetes.io/service-name=python-api` shows no endpoints: the Service's selector says `app: python-apii`. Fix: `kubectl apply -f kubernetes/python-api/service.yaml`.

2. **go-status rollout stuck.** `kubectl get pods -l app=go-status` shows a new Pod in `ImagePullBackOff` next to the old, still-working Pod; `kubectl describe` says the image `go-status:1.0.1` cannot be pulled (it was never built). The status board itself still works: the rolling update protected it. Fix: `kubectl rollout undo deployment/go-status`.
3. **java-api rollout stuck.** A restart was triggered after the ConfigMap changed. The new java-api Pod never becomes Ready; its `/ready` (and the log) report that the database `bookshp` does not exist. The ConfigMap's `DB_NAME` is misspelled. Fix: `kubectl apply -f kubernetes/config/bookshop-config.yaml`, then `kubectl rollout restart deployment/java-api`. (Other services did not restart, so they still use the old, correct value: a ConfigMap change is a time bomb until the next restart.)

```
kubectl apply -f kubernetes/python-api/service.yaml
kubectl rollout undo deployment/go-status
kubectl apply -f kubernetes/config/bookshop-config.yaml
kubectl rollout restart deployment/java-api
kubectl rollout status deployment/go-status --timeout=240s
kubectl rollout status deployment/java-api --timeout=240s
```

```
bash capstone/verify.sh
```

```
...
PASS every Deployment Available again afterwards

22 passed, 0 failed
```

What you can say now

"I know how to take applications written in different languages, package them as Docker containers, run them together using Docker Compose, understand their dependencies and networking, and deploy those applications to Kubernetes using the appropriate Kubernetes resources."

When you are done: [kubernetes/cleanup.md](#).

Glossary

Every term used in this course, in plain words. The link points to where it is explained or used.

Term	Meaning
.dockerignore	A list of files Docker must not send to the build. Keeps images small and secrets out of them. docs/03
.env file	A file of <code>NAME=value</code> lines Compose reads for variables in <code>docker-compose.yml</code> . Holds secrets locally; never committed. compose/README.md
Allocated resources	The sum of all requests and limits of the Pods on a node, shown by <code>kubectl describe node</code> . kubernetes/13
APP_KEY	Laravel's encryption key, needed to sign sessions and cookies. Here it comes from a Secret. applications/laravel-admin
Base64	A way to write any bytes as text. Kubernetes stores Secret values in base64: an encoding, not encryption. kubernetes/10
Build context	The folder Docker sends to the builder; everything a <code>COPY</code> can use. docs/03
Build stage	The first part of a multi-stage Dockerfile, with compilers and package managers, whose results are copied into a small final image. docs/03
ClusterIP	The default Service type: a virtual IP reachable only inside the cluster. docs/11
ConfigMap	A Kubernetes object holding non-secret configuration as key-value pairs, injected as environment variables or files. docs/07
Container	A running instance of an image: an isolated process with its own filesystem, network and limits. docs/03
containerPort	The port a container listens on, declared in the Pod template; the Service's <code>targetPort</code> must match it. troubleshooting 03
Contract (service contract)	The rules every Bookshop service follows: PORT, /health, /ready, environment configuration, stdout logs, SIGTERM. docs/CONTRACT.md
CrashLoopBackOff	A Pod status: the container keeps exiting and Kubernetes waits longer before each restart. The cause is in <code>kubectl logs --previous</code> . troubleshooting 05
CronJob	A Kubernetes object that creates a Job on a schedule (cron syntax), like report-worker every 10 minutes. docs/14
depends_on	A Compose setting for start order; with a <code>condition</code> it waits for health or completion. Kubernetes has no equivalent. docs/05
Deployment	A Kubernetes object that keeps a number of identical Pods running and replaces them with rolling updates. kubernetes/02
Distroless	A base image with only what a program needs to run: no shell, no package manager. Used by go-status. kubernetes/05
DNS (cluster DNS)	The name service that resolves Service names such as <code>node-api</code> to their IPs inside the cluster. docs/11

Term	Meaning
Docker Compose	A tool that runs several containers from one YAML file on one machine. compose/README.md
Dockerfile	The recipe for an image: base image, files, commands, user, port, start command. docs/03
Endpoints / EndpointSlice	The list of Ready Pod IPs and ports behind a Service. Empty endpoints mean no traffic. troubleshooting 04
Environment variable	A named value given to a process at start; the way every service here is configured. docs/CONTRACT.md
ErrImagePull / ImagePullBackOff	Pod statuses when a node cannot pull the image: wrong name, missing tag, or no access. troubleshooting 01
Events	Short messages Kubernetes records about objects: scheduling, pulls, probe failures, kills. <code>kubect1 get events</code> . docs/13
Exit code 137	128 + 9: the process was killed with SIGKILL, usually OOMKilled at the memory limit. kubernetes/13
Express	A web framework for Node.js, used by node-api. applications/node-api
FastAPI	A Python web framework, used by python-api, served by uvicorn. applications/python-api
GHCR	GitHub Container Registry; CI publishes this course's images there. docs/04
Graceful shutdown	Stopping cleanly on SIGTERM (finish requests, close connections) instead of being killed. docs/CONTRACT.md
Headless Service	A Service without a cluster IP (<code>clusterIP: None</code>); its name resolves directly to Pod IPs. Used for PostgreSQL. docs/10
Health check (Compose)	A command Compose runs to decide whether a container is healthy; maps to Kubernetes probes. docs/05
Helm	The package manager for Kubernetes; installs Traefik in this course. kubernetes/00
Image	A packaged, read-only filesystem plus start settings, from which containers are created. docs/04
Image tag	The version part of an image name (<code>node-api:1.0.0</code>). Versioned, immutable tags make rollbacks possible. docs/04
imagePullPolicy	When a node pulls an image: <code>IfNotPresent</code> (use the local one if there) or <code>Always</code> . docs/04
Ingress	A Kubernetes object with HTTP routing rules (host, path → Service). kubernetes/09
Ingress controller	The program that carries out Ingress rules; Traefik in this course. docs/11
JavaScript vs Node.js	JavaScript is the language; Node.js is a runtime that runs JavaScript outside the browser. docs/02
Job	A Kubernetes object that runs Pods until a task completes successfully, like the Laravel migrations. docs/14

Term	Meaning
kind	"Kubernetes in Docker": runs Kubernetes nodes as Docker containers on your computer. kubernetes/00
kind load docker-image	Copies an image from your Docker into the kind nodes' image store. kubernetes/02
kubectl	The command-line client for the Kubernetes API. kubernetes/README.md
Label / selector	Key-value tags on objects, and the queries that select them; how Services and Deployments find their Pods. labs/05
Laravel	A PHP web framework, used by laravel-admin. applications/laravel-admin
Limits	The maximum CPU (throttled above it) and memory (killed above it) a container may use. docs/09
Liveness probe	A periodic check; when it fails repeatedly, the kubelet restarts the container. docs/08
Multi-container Pod	A Pod with several containers that share network and lifecycle, like nginx + PHP-FPM. docs/15
Multi-stage build	A Dockerfile with several <code>FROM</code> stages; only the last one becomes the shipped image. docs/03
Namespace	A named group of Kubernetes objects; everything of this platform is in <code>bookshop</code> . kubernetes/01
Named volume	Storage managed by Docker that outlives containers, like <code>pgdata</code> . compose/README.md
Network (Compose)	A virtual network for containers, with DNS by service name; separate networks isolate services. docs/05
nginx	A web server and reverse proxy; serves the React files and fronts PHP-FPM. applications/frontend
NodePort	A Service type that opens the same port on every node; Traefik listens on 30080 here. docs/11
Non-root	Running a container as an ordinary user, enforced with <code>runAsNonRoot</code> . docs/03
OOMKilled	The container went over its memory limit and the kernel killed it. kubernetes/13
Pending	A Pod that has no node yet (no resources, or an unbound volume claim); reason in the events. troubleshooting 09
PersistentVolume (PV)	A piece of storage in the cluster, created by a provisioner for a claim. docs/10
PersistentVolumeClaim (PVC)	A request for storage by a Pod; bound to a PersistentVolume. kubernetes/12
PHP-FPM	The process manager that runs PHP code for a web server over FastCGI (port 9000). docs/15
Pod	The smallest unit Kubernetes runs: one or more containers with one IP. kubernetes/02
port-forward	<code>kubectl port-forward</code> : a temporary tunnel from your computer to a Service or Pod. kubernetes/02

Term	Meaning
Profile (Compose)	A label that keeps a service out of <code>up</code> unless asked for, like <code>jobs</code> for report-worker. docs/05
Readiness probe	A periodic check; while it fails, the Pod gets no traffic from its Services. docs/08
Registry	A server that stores images (Docker Hub, GHCR, ECR). docs/04
ReplicaSet	The object a Deployment uses to keep N Pods of one template; old ones are kept for rollbacks. kubernetes/14
Replicas	The number of identical Pods a Deployment keeps running. docs/12
Requests	The CPU and memory the scheduler reserves for a container. docs/09
Revision	A numbered version of a Deployment's Pod template, listed by <code>kubectl rollout history</code> . kubernetes/14
Rolling update	Replacing Pods a few at a time, each new one first passing readiness, so there is no downtime. docs/12
Rollback (rollout undo)	Returning a Deployment to a previous revision. kubernetes/14
Runtime configuration	Settings read when a container starts (like the frontend's <code>/config.js</code>), so one image fits every environment. kubernetes/03
Secret	A Kubernetes object for sensitive values, stored base64-encoded and protected by access control. docs/07
Service	A stable name and virtual IP that load-balances to the Ready Pods matching its selector. docs/11
Service discovery	Finding other services by name (Compose DNS, Kubernetes Service DNS) instead of by IP. docs/01
SIGTERM	The signal that asks a process to stop; sent by <code>docker stop</code> and by Kubernetes before killing a container. docs/CONTRACT.md
Spring Boot	A Java framework for applications that run as one executable JAR, used by java-api. applications/java-api
Startup probe	A check that gives slow starters time; liveness and readiness wait until it succeeds. kubernetes/06
StatefulSet	A workload with stable Pod names and one volume claim per Pod; used for PostgreSQL. docs/10
StorageClass	A kind of storage the cluster can provision; kind's default is <code>standard</code> . troubleshooting 09
targetPort	The Pod port a Service forwards to. troubleshooting 10
Traefik	The ingress controller used in this course, installed with Helm. kubernetes/00
Vite	The build tool that turns the React source into static files. applications/frontend

Interview questions

25 questions that come up in interviews for junior and intermediate DevOps, platform and cloud roles. Answer each one out loud first, then open the model answer. Each answer links to the lesson where you did it yourself: in an interview, "I did this in a lab, and here is what I saw" beats any memorised definition.

Containers and Dockerfiles



1. How would you containerise an application written in a language you don't know?

Find out four things: how it is built, how it is started, which port it listens on, and how it is configured. Then pick the official runtime image, copy the dependency manifest before the source so the dependency layer is cached, run as a non-root user, and start the process in exec form so it receives signals. A health endpoint and environment-based configuration make it deployable anywhere. ([docs/CONTRACT.md](#))



1. What is a multi-stage build and when is it worth it?

A Dockerfile with several `FROM` stages: early stages compile or install with full toolchains, the last stage copies only the results into a small runtime image. It is worth it whenever a build step exists: go-status shrank from a 381 MB build image to 16.4 MB, the frontend from 334 MB to 81.8 MB. Fewer files also means fewer vulnerabilities. ([docs/03](#))



1. Why is the order of Dockerfile instructions important?

Each instruction is a cached layer; a change invalidates that layer and every layer after it. Copying the dependency manifest (`package-lock.json` , `requirements.txt` , `pom.xml` , `go.mod` , `composer.lock`) and installing before copying the source means code changes do not reinstall dependencies. Builds go from minutes to seconds. ([docs/03](#))



1. Why should images use versioned tags instead of latest?

`latest` is a moving pointer: you cannot tell which build a node runs, different nodes may run different builds, and there is no previous version to roll back to. A versioned, immutable tag such as `node-api:1.0.0` always means the same image, which makes deployments predictable and `rollout undo` meaningful. ([docs/04](#))



1. JavaScript and Node.js: what is the difference, and why does it matter for deployment?

JavaScript is a language; Node.js is a runtime that executes it outside the browser. React code runs in the user's browser, so its container only serves files; node-api is a long-running server (a Deployment); report-worker runs once and exits (a Job or CronJob). The same language ends up in three different Kubernetes workload types. ([docs/02](#))

Docker Compose



1. What does `depends_on` guarantee in Docker Compose?

Without a condition, only that the other container was started first, not that its application is ready. With `condition: service_healthy` Compose waits for the health check, and with `service_completed_successfully` for a one-off task such as a migration to finish. It only orders startup; it does not react to later failures. ([compose/README.md](#))



1. How do containers in Compose find each other, and how can you isolate them?

Compose creates networks with built-in DNS, so a container reaches another by its service name (`http://java-api:8080`). Services on different networks cannot even resolve each other: in this lab the frontend is not on the backend network, so `postgres` is an unknown name for it. Networks are the first isolation layer. ([compose/README.md](#))



1. What is the difference between `docker compose down` and `down -v`?

`down` removes containers and networks; named volumes survive, so the database keeps its data. `down -v` also removes the volumes and therefore the data. In this lab a user added before `down` was still there after `up` ; `down -v` is treated as a destructive command. ([compose/README.md](#))

From Compose to Kubernetes



1. How do the parts of a Compose file map to Kubernetes resources?

A service becomes a Deployment (a StatefulSet for a database, a Job or CronJob for one-off work); ports become a Service inside the cluster and an Ingress from outside; environment becomes ConfigMaps and Secrets; volumes become PersistentVolumeClaims; health checks become probes. `depends_on` has no equivalent: readiness probes and retrying applications replace it. ([docs/06](#))



1. Why not simply convert a Compose file with a tool?

Converters produce a starting point, but the decisions that make a deployment reliable are not in the Compose file: which probe checks what, how much CPU and memory to request, which values are secret, which workload type fits, how storage survives. You still have to understand and review the YAML, so you need to know the mapping anyway. ([docs/06](#))



1. How does a kind cluster get your locally built images?

The kind nodes are containers with their own image store, separate from your Docker. `kind load docker-image` copies an image into every node; with `imagePullPolicy: IfNotPresent` the node uses it. Forgetting this makes the node try to pull from Docker Hub and fail with `ImagePullBackOff`; on real clusters you push to a registry instead. ([troubleshooting 02](#))

Workloads



1. When do you use a Deployment, a StatefulSet, a Job and a CronJob?

A Deployment for stateless processes that run forever (the APIs, the frontend). A StatefulSet when each Pod needs a stable name and its own storage (PostgreSQL). A Job for a task that must run to completion once (migrations). A CronJob for a Job on a schedule (the report worker). The choice depends on the program's lifecycle, not its language. ([docs/14](#))



1. When do two containers belong in the same Pod?

When they always run and scale together, share their lifecycle and talk over localhost: nginx in front of its own PHP-FPM is the classic example. Containers in a Pod share the network namespace and can share volumes. If the two parts could scale independently, they should be separate Deployments with a Service between them. ([docs/15](#))



1. How should database migrations run on Kubernetes?

Once per release, before the new version serves traffic, typically as a Job (or a pipeline step / Helm hook). Running them at application startup in every replica can run them concurrently and corrupt or lock the schema. In this lab the `laravel-migrate` Job runs `migrate --force --seed` and must complete before the application is rolled out. ([kubernetes/07](#))

Networking



1. What is the difference between a Service and an Ingress?

A Service gives a set of Pods a stable name and virtual IP and load-balances to the Ready ones, mainly inside the cluster. An Ingress is a set of HTTP rules (host and path to Service) for traffic from outside, carried out by an ingress controller such as Traefik. One Ingress can route a whole platform. ([docs/11](#))



1. A Service returns 503. How do you debug it?

Check its endpoints first. Empty endpoints mean either the selector matches no Pod labels or the Pods are not Ready; compare the selector with `--show-labels` and look at readiness events. If endpoints exist, check the ports: the Service's `targetPort` must match the port the application really listens on. ([troubleshooting 04](#), [troubleshooting 10](#))



1. Why do the same service names work in Compose and in Kubernetes?

Both provide DNS for service discovery: Compose resolves service names on its networks, Kubernetes resolves Service names in the namespace (`node-api` , or fully `node-api.bookshop.svc.cluster.local`). Keeping the names identical lets the same images and configuration values work in both. ([docs/01](#))

Configuration and Secrets



1. Are Kubernetes Secrets encrypted?

Not inherently: the values are base64-encoded, which anyone can decode. They are protected by RBAC, by never being in images or Git, by being mounted only into Pods that reference them, and, if the cluster is configured for it, by encryption at rest in etcd. External secret stores add rotation and auditing. ([docs/07](#))



1. You changed a ConfigMap and nothing happened. Why?

Values injected as environment variables are read when the container starts, so running Pods keep the old values. A `kubectl rollout restart` creates new Pods that read the new values. Mounted ConfigMap files update after a delay, but the application must re-read them. ([kubernetes/10](#))

Probes and resources



1. Explain liveness, readiness and startup probes.

Liveness asks "should this container be restarted?"; readiness asks "should this Pod receive traffic now?"; a startup probe protects slow starters by delaying the other two until it succeeds. In this lab `/health` (process alive) is the liveness check, `/ready` (database reachable) the readiness check, and `java-api` has a startup probe because the JVM needed 9.3 s at 0.5 CPU. ([docs/08](#))



1. What happens to your APIs when the database goes down, if the probes are designed well?

Readiness fails, so the Pods leave their Services and the Ingress answers 503 quickly instead of timing out; liveness keeps passing, so no Pod restarts. When the database returns, readiness passes and traffic resumes without any restart. A liveness probe that checked the database would cause a restart storm instead. ([kubernetes/11](#))



1. What are requests and limits, and what happens when a limit is exceeded?

Requests are what the scheduler reserves on a node; limits are the maximum. Above the CPU limit a container is throttled (slower); above the memory limit it is killed (`OOMKilled`, exit code 137). Requests should match real usage, limits leave headroom; JVMs need their heap sized to fit the limit. ([docs/09](#))

Operations and troubleshooting



1. How does a rolling update protect you from a bad release?

New Pods are added and old ones removed step by step, and each new Pod must pass its readiness probe first. If the new version never becomes Ready (crash, OOM, wrong probe, missing image), the rollout stalls and the old Pods keep serving. In this lab a memory limit that was too small caused no outage. The stalled rollout must still be noticed and fixed. ([kubernetes/13](#))



1. How do you roll back a Deployment, and what makes it possible?

`kubectl rollout undo` returns to the previous revision: the Deployment keeps old ReplicaSets at zero replicas and scales one up again. It works because the old image still exists under its version tag. Afterwards, update the manifest in Git, or the next apply re-deploys the bad version. ([kubernetes/14](#))



1. A Pod is not working. What is your first command, depending on its status?

`ImagePullBackOff` → `kubectl describe pod` (the event names the image). `CrashLoopBackOff` → `kubectl logs --previous`. `Running but 0/1` → probe events and the readiness endpoint. `Ready but unreachable`

→ `kubect1 get endpoints` and the ports. Pending → events (resources or volume claims). The status column decides where to look. ([docs/13](#), [troubleshooting](#))